

Senior Officer (IT) — Written Exam Master Guide

Bangladesh Bank / Bankers' Selection Committee (BSC) • Job ID 25104 •
Written Exam 25 July 2026

Contents

- 1. Computer Architecture & Digital Electronics** — number systems, 2's complement, logic gates, K-map, MUX, half/full adder, CPU & memory hierarchy.
- 2. Operating Systems** — process vs thread, CPU scheduling (with Gantt maths), deadlock/livelock, Banker's algorithm, virtual memory, paging vs segmentation, disk maths.
- 3. Database Management** — ER diagrams, normalization (1NF–3NF), SQL queries, indexing, stored procedures & triggers, ACID, RAID.
- 4. Networking & Security** — OSI model, error detection (CRC), protocols & ports, network devices, subnetting maths, bandwidth & multiplexing, CIA triad, hashing, symmetric vs asymmetric cryptography, cyber attacks, firewall/IDS/IPS, VPN, steganography, OTP.
- 5. Programming** — pointers, recursion, loops, arrays, **22 complete C programs**, OOP four pillars, overloading vs overriding, MVC.
- 6. Data Structures & Algorithms** — Big-O, stack/queue, infix–postfix, trees & BST, heaps, sorting, BFS/DFS, Prim's & Kruskal's, Dijkstra, Huffman coding.
- 7. Software Engineering** — SDLC models, testing, API/REST, Git.
- 8. Emerging Technologies** — cloud service & deployment models, virtualization/VM/containers, data-centre infrastructure, backup, DR (RTO/RPO), SLA.
- 9. Last-48-Hours Rapid Revision Sheet** — one-line reminders for every topic + exam-day strategy.
- 10. Predicted Question Bank** — 50 likely questions with answers.

How to use this guide

This guide targets the **Subject-Related** part of your written paper — the section that decides your result.

Written section	Marks
Focus Writing (Bangla)	10
Focus Writing (English)	10
Translation (Bangla → English)	10
Translation (English → Bangla)	10
General Knowledge	10
Subject-Related — 15 questions × 10 marks	150
Total	200

150 of the 200 marks are pure IT subject questions — fifteen 10-mark questions. A 10-mark answer is a *mini-essay*: the examiner expects (1) **a clear definition**, (2) **an easy explanation**, (3) **a worked example or labelled diagram**, and (4) — **for a bank — a line on how it applies to banking**. One-line answers score 2–3 marks; a full structured answer scores 8–10.

For every topic below you get:

- **The idea in plain words** (with an everyday analogy so it sticks).
- **How it works**, step by step.
- **Worked examples** you can reproduce exactly in the exam.
- **A clean black-and-white diagram** wherever a picture earns marks.
- **★ Predicted questions** — written the way they will actually appear, with model answers. There is also a **50-question predicted question bank** at the end (Section 10).

How past BSC / Bank IT written papers are set (verified pattern)

Across the *Combined Bank Senior Officer (IT/ICT)* papers and recent single-bank IT written tests, these items repeat almost every time:

1. **Number system / 2's-complement** conversion or arithmetic.
2. **CPU scheduling** — draw the Gantt chart, compute average waiting & turnaround time.
3. **Subnetting** — “given $192.168.x.0/24$, make N subnets: find mask, hosts, network & broadcast.”
4. **Normalization** (1NF → 2NF → 3NF) and/or an **ER diagram**.
5. **SQL query** writing, **ACID** with examples, and **indexing**.
6. **Write a complete program** (C / C++ / Python) — prime, factorial, Fibonacci, palindrome, sorting, largest-in-array.
7. **DSA** — infix → postfix, trace a sorting algorithm, BST, Dijkstra, BFS/DFS.
8. **OSI model, symmetric vs asymmetric cryptography, hashing, firewall / IDS / IPS.**

9. **OOP** — four pillars with examples; overloading vs overriding.
10. **Cloud** service & deployment models; **RAID**; **SDLC**; **backup / DR / SLA**.

Because this is a **bank**, examiners frame answers around **Core Banking Systems (CBS)**, **ATM transaction logs**, and **digital-banking security** — so add that angle wherever it fits; it earns marks.

The 4-step answer formula (use it for every 10-marker)

Define — Explain simply — Show an example/diagram — Add the banking relevance. Always draw the diagram if the topic has one — a clear labelled figure alone is worth 2–3 marks. For any math, **show every step** and **box the final answer**; method carries marks even if a number slips.

1. Computer Architecture & Digital Electronics

1.1 Number System Conversions

The idea. A *number system* is just a way of counting using a fixed set of symbols. The **base** tells you how many symbols you have and the “value of each place”.

- **Decimal (base 10):** digits 0–9 — what humans use.
- **Binary (base 2):** digits 0,1 — what computers use (a wire is either ON=1 or OFF=0).
- **Octal (base 8):** digits 0–7.
- **Hexadecimal (base 16):** digits 0–9 then A,B,C,D,E,F (A=10 ... F=15) — a short way to write long binary.

Binary place values (memorize left-to-right): 128 64 32 16 8 4 2 1. Each place is double the one to its right.

Decimal → Binary (divide-by-2 method)

Keep dividing by 2, write the remainder each time, then read the remainders **bottom to top**.

Example — convert 45 to binary:

```
45 / 2 = 22 remainder 1 (LSB)
22 / 2 = 11 remainder 0
11 / 2 = 5 remainder 1
5 / 2 = 2 remainder 1
2 / 2 = 1 remainder 0
1 / 2 = 0 remainder 1 (MSB)
```

Read remainders upward: 1 0 1 1 0 1

45 = (101101)₂

Check: 32 + 8 + 4 + 1 = 45

Binary → Decimal (multiply-by-place-value)

Multiply each bit by its place value and add:

```
1 0 1 1 0 1
32 0 8 4 0 1  -> 32 + 8 + 4 + 1 = 45
```

Binary ↔ Octal ↔ Hex (grouping shortcut)

- **Binary → Octal:** group bits in **3s** from the right (because $2^3 = 8$).
- **Binary → Hex:** group bits in **4s** from the right (because $2^4 = 16$).

Binary: 1 0 1 1 0 1

Octal : group 3s -> 101 101 -> 5 5 -> (55)₈

Hex : group 4s -> 0010 1101 -> 2 D -> (2D)₁₆

- **Hex → Binary:** replace each hex digit with its 4-bit code. 2D → 0010 1101.
- **Octal → Binary:** replace each octal digit with its 3-bit code. 55 → 101 101.

Hex digit table (learn it): A=1010=10, B=1011=11, C=1100=12, D=1101=13, E=1110=14, F=1111=15.

Fraction conversion (decimal fraction → binary)

Multiply the fractional part by 2 repeatedly; read the **integer parts top-to-bottom**.

```
0.625 x 2 = 1.25  -> 1
0.25  x 2 = 0.50  -> 0
0.50  x 2 = 1.00  -> 1   (stop)
0.625 = (0.101)2
```

★ **Predicted question (10 marks):** “Convert (a) 156 to binary, octal, hex; (b) $(11010110)_2$ to decimal, octal, hex; (c) 0.375 to binary.” **Answer:** (a) $156 = 10011100_2 = 234_8 = 9C_{16}$. (b) $11010110_2 = 214 = 326_8 = D6_{16}$. (c) $0.375 \times 2 = 0.75 - 0$, $0.75 \times 2 = 1.5 - 1$, $0.5 \times 2 = 1.0 - 1 \Rightarrow 0.011_2$.

1.2 2's Complement (how computers store negatives)

The idea. Computers have no minus sign — only 0s and 1s. **2's complement** is the trick that lets the same adder circuit handle both positive and negative numbers, and it gives exactly **one zero** (unlike sign-magnitude, which has +0 and -0).

The leftmost bit is the sign bit: 0 = positive, 1 = negative.

How to find 2's complement — “flip and add one”

1. Write the positive number in binary at a fixed width (say 8 bits).
2. **1's complement** — flip every bit ($0 \leftrightarrow 1$).
3. **Add 1** — the result is 2's complement (the negative form).

Example — represent -6 in 8 bits:

```
+6           = 0000 0110
1's complement = 1111 1001   (flip every bit)
add 1        = 1111 1010   -> -6 in 2's complement
```

Subtraction becomes addition

$A - B = A + (2\text{'s complement of } B)$. Compute **5 - 6** in 8 bits:

```
5           = 0000 0101
-6          = 1111 1010   (2's complement of 6, from above)
add         = 1111 1111   -> MSB = 1, so negative
value: flip(1111 1111)=0000 0000, +1 = 0000 0001 = 1  -> answer = -1
```

Range and overflow

- **n-bit 2's complement range:** $-2^{n-1} \dots +2^{n-1} - 1$. For 8 bits: **-128 ... +127**.
- **Overflow** happens when two same-sign numbers add to the wrong sign (positive + positive = negative).

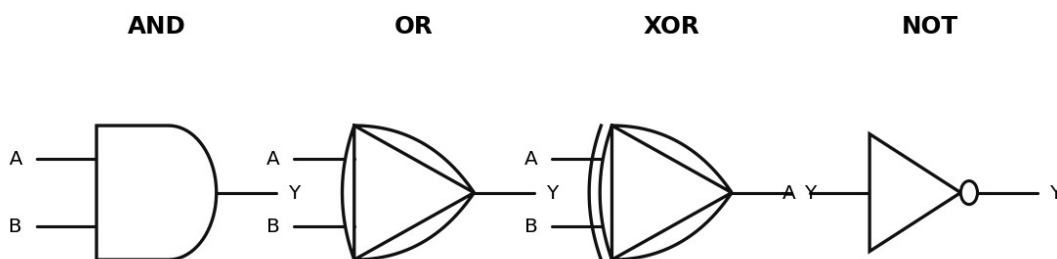
Memory hook: *Flip and add one.* **1's complement = just flip; 2's complement = flip then +1.**

★ **Predicted question (10 marks):** “(a) Represent -45 in 8-bit 2’s complement. (b) Perform $30 - 45$ in 8-bit 2’s complement and verify.” **Answer:** (a) $+45 = 0010\ 1101$; flip $\rightarrow 1101\ 0010$; $+1 \rightarrow 1101\ 0011 = -45$. (b) $30 = 0001\ 1110$; add $1101\ 0011 \rightarrow 1111\ 0001$. MSB=1 $\Rightarrow$ negative; magnitude = flip($1111\ 0001$)= $0000\ 1110$, $+1 = 0000\ 1111 = 15 \Rightarrow$ result -15 . Check: $30 - 45 = -15$.

1.3 Logic Gates & Boolean Algebra

The idea. A **logic gate** is a tiny electronic circuit that takes 1-or-0 inputs and produces a 1-or-0 output following a fixed rule. Combine gates and you can build everything — adders, memory, whole CPUs.

Basic Logic Gate Symbols



Basic logic gate symbols

Gate	Expression	Output = 1 when...	Truth (A,B)
AND	$A \cdot B$	both inputs are 1	00→0, 01→0, 10→0, 11→1
OR	$A + B$	at least one input is 1	00→0, 01→1, 10→1, 11→1
NOT	A'	input is 0 (inverter)	0→1, 1→0
NAND	$(A \cdot B)'$	NOT of AND	00→1, 01→1, 10→1, 11→0
NOR	$(A + B)'$	NOT of OR	00→1, 01→0, 10→0, 11→0
XOR	$A \oplus B$	inputs are different	00→0, 01→1, 10→1, 11→0
XNOR	$(A \oplus B)'$	inputs are same	00→1, 01→0, 10→0, 11→1

Universal gates — NAND and NOR. *Universal* means any logic circuit can be built using **only** NAND gates (or only NOR gates). This matters because chip makers can mass-produce a single gate type. *Example:* a NOT gate = a NAND with both inputs tied together.

Boolean laws you must know (used to simplify circuits)

- **Identity:** $A + 0 = A$, $A \cdot 1 = A$
- **Null:** $A + 1 = 1$, $A \cdot 0 = 0$
- **Complement:** $A + A' = 1$, $A \cdot A' = 0$
- **Idempotent:** $A + A = A$, $A \cdot A = A$
- **Absorption:** $A + A \cdot B = A$, $A \cdot (A + B) = A$
- **De Morgan's (exam favourite):** $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$

Worked simplification: $A'BC + A'B'C + ABC$. Group first two: $A'C(B + B') = A'C$. Left with $A'C + ABC = C(A' + AB) = C(A' + B)$. **Answer: $C(A' + B)$.**

1.4 K-Map (Karnaugh Map)

The idea. A K-map is a picture that simplifies Boolean expressions **faster than algebra**. You place the 1s of the function on a grid arranged in **Gray-code order** (only one bit changes between neighbouring cells) so that **adjacent 1s can be grouped**. Each big group of 1s becomes one small product term.

4-variable K-map: group adjacent 1s

		CD				
		00	01	11	10	
AB	00	1	0	0	1	Groups of 1s → simplified terms
	01	1	0	0	1	
	11	1	1	1	1	
	10	1	0	0	1	

4-variable K-map: grouping adjacent 1s

Rules (memorize):

1. Fill the map with the function's 1s (from its truth table / minterms).
2. Group 1s into rectangles of size **1, 2, 4, 8 or 16** (powers of two only).
3. Make groups **as large as possible** (bigger group $\Rightarrow$ simpler term) and use as **few** groups as possible.

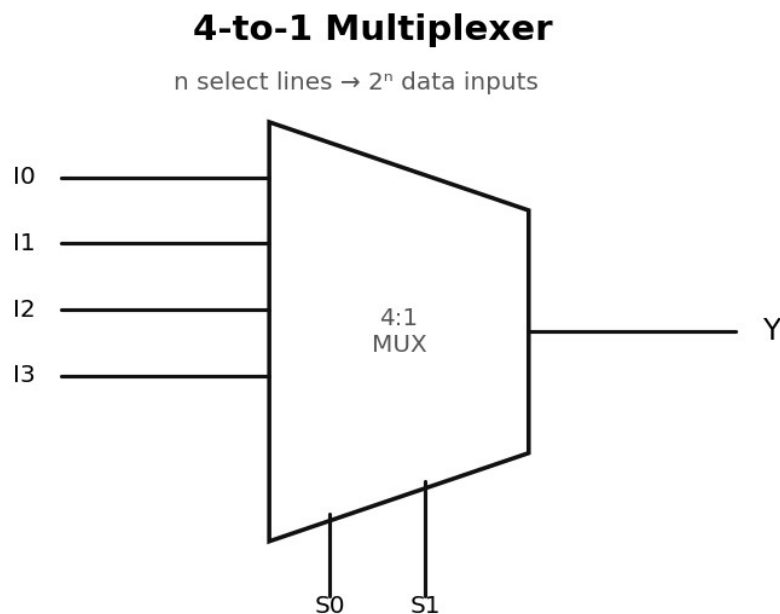
- Groups may **overlap** and may **wrap around** the edges (top→bottom, left→right).
- In each group, a variable that **changes** is eliminated; a variable that stays 0 appears complemented (A'), that stays 1 appears as A .

Worked 2-variable example: $F = A'B + AB$. The two cells share B while A changes $\rightarrow F = B$.

Worked 3-variable example: $F(A, B, C) = \Sigma(1, 3, 5, 7)$. Every one of these minterms has $C = 1$, so all four group together $\rightarrow F = C$.

1.5 Multiplexer (MUX) & Demultiplexer

The idea. A **multiplexer** is a *data selector* — many inputs go in, the **select lines** choose exactly one of them to appear at the single output. Think of a **railway switch operator**: many incoming tracks, one outgoing track, and a control decides which train passes.



4-to-1 multiplexer

- n select lines** $\rightarrow$ **2^n data inputs** $\rightarrow$ **1 output**. So a 4-to-1 MUX needs **2** select lines; an 8-to-1 needs **3**.
- A **demultiplexer (DEMUX)** is the reverse: **1 input** $\rightarrow$ **2^n outputs**; the select lines pick which output receives the input.
- A **decoder** turns an n -bit code into exactly one active line out of 2^n (used to select memory locations). An **encoder** does the opposite.

Uses: sharing one communication line among many sources, routing data inside the CPU, memory addressing.

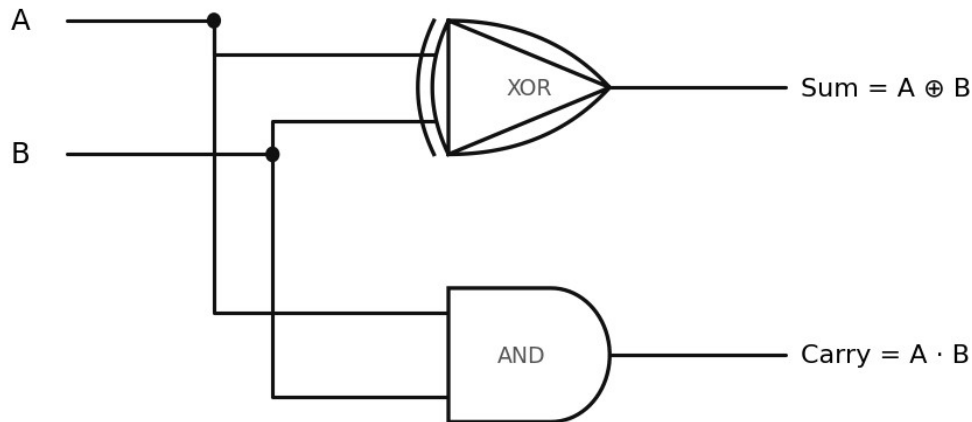
★ **Predicted question:** “How many select lines are needed for a 16-to-1 MUX, and what does a MUX do?” $\rightarrow$ **4 select lines** ($2^4 = 16$). A MUX selects one of many inputs to pass to a single output based on the select lines.

1.6 Adders — Half Adder & Full Adder

Why they matter: addition is the core operation of the ALU (Arithmetic Logic Unit). Every subtraction, multiplication and address calculation is built on adders.

Half Adder — adds two bits

Half Adder (gate-level circuit)



Half adder — gate-level circuit

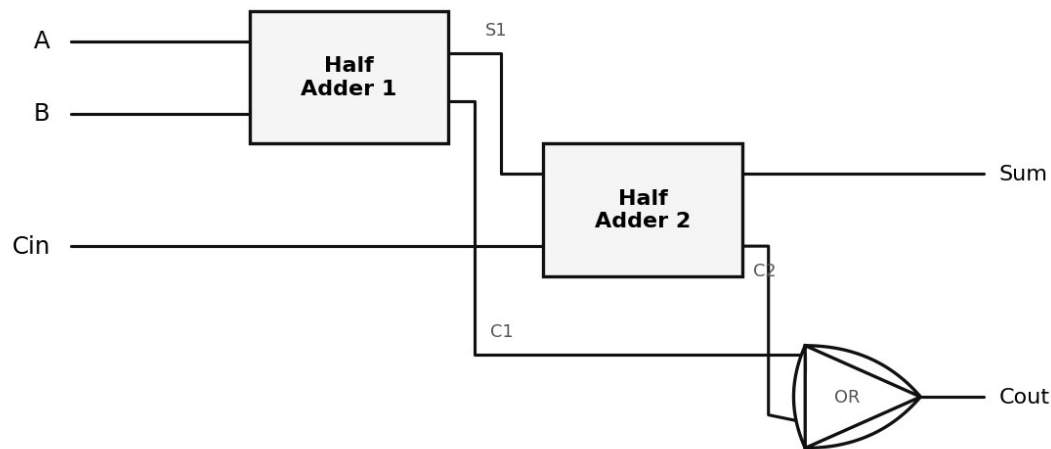
Inputs A, B → outputs **Sum** and **Carry**:

Sum = A XOR B → 1 when A, B differ
Carry = A AND B → 1 only when both are 1

Truth table: 0+0 = 00, 0+1 = 01, 1+0 = 01, 1+1 = 10. **Limitation:** it cannot accept a carry coming *in* from a previous bit, so it can only add the least-significant bit of a multi-bit number.

Full Adder — adds three bits (A, B, and Carry-in)

Full Adder (built from two Half Adders)



Full Adder = two Half Adders + one OR gate

Full adder — two half adders + OR gate

$$\text{Sum} = A \text{ XOR } B \text{ XOR } \text{Cin}$$

$$\text{Cout} = A.B + \text{Cin}.(A \text{ XOR } B)$$

A full adder is built from **two half adders plus one OR gate**. To add multi-bit numbers you chain n full adders so the **carry ripples** from the least-significant to the most-significant bit — a **ripple-carry adder**, the heart of the ALU.

★ **Predicted question (10 marks):** “Design a full adder. Give its truth table, the Boolean expressions for Sum and Carry, and a circuit diagram.” → Provide the 3-input truth table (8 rows), the two expressions above, and the diagram (two half adders feeding an OR gate).

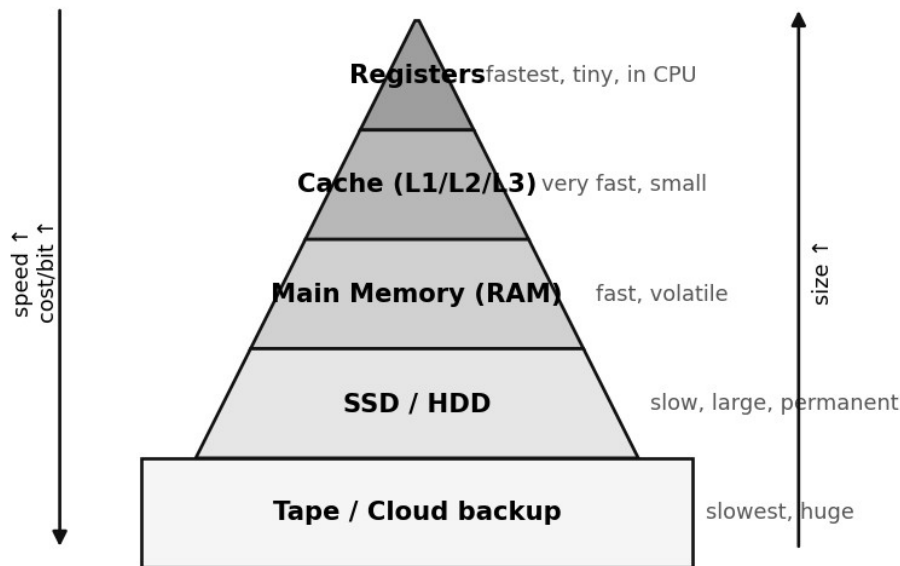
1.7 CPU & Architecture — high-yield facts

The idea. The **CPU (Central Processing Unit)** is the “brain”. It repeats one simple cycle billions of times per second.

- **Instruction cycle (Fetch–Decode–Execute):** *Fetch* the instruction from memory → *Decode* what it means → *Execute* it → (write back the result).
- **CPU parts:** **ALU** (arithmetic/logic), **Control Unit** (directs everything), **Registers** (tiny ultra-fast storage inside the CPU).
- **System bus = Data bus** (carries data) + **Address bus** (carries the memory address) + **Control bus** (carries control signals such as read/write).

Memory hierarchy (fast + small on top, slow + huge at bottom)

Memory Hierarchy



Memory hierarchy pyramid

Going **down** the pyramid: speed decreases, cost-per-bit decreases, capacity increases. The CPU looks in registers first, then cache, then RAM, then disk. **Cache** keeps recently/frequently used data close to the CPU (locality of reference).

Must-know contrasts

- **Von Neumann vs Harvard:** Von Neumann keeps program *and* data in the **same** memory (simpler, but a single-bus bottleneck). Harvard uses **separate** instruction and data memories/buses (faster; used in DSPs, microcontrollers).
- **RISC vs CISC:** RISC = few, simple, fixed-length instructions (ARM — phones). CISC = many complex instructions (x86 — PCs).
- **Pipelining:** overlap fetch/decode/execute of consecutive instructions (like a laundry assembly line) to raise throughput.
- **Volatile vs non-volatile:** RAM loses data on power-off (volatile); ROM / SSD / HDD keep it (non-volatile).
- **SRAM vs DRAM:** SRAM is fast, used in cache, needs no refresh; DRAM is denser, cheaper, needs refresh, used as main memory.

2. Operating Systems

What an OS is. The **Operating System** is the software layer between the hardware and your programs. It manages the CPU, memory, files, and devices, and gives programs a clean interface so they don't have to talk to hardware directly. Main jobs: **process management, memory management, file management, I/O management, security.**

2.1 Process vs Thread

- A **process** is a *program in execution*. It has its **own private memory** (code, data, heap, stack), a process ID (PID), and its own resources.
- A **thread** is the **smallest unit of execution** *inside* a process. Threads of the same process **share** the code, data and heap, but each thread has its **own stack and registers**.

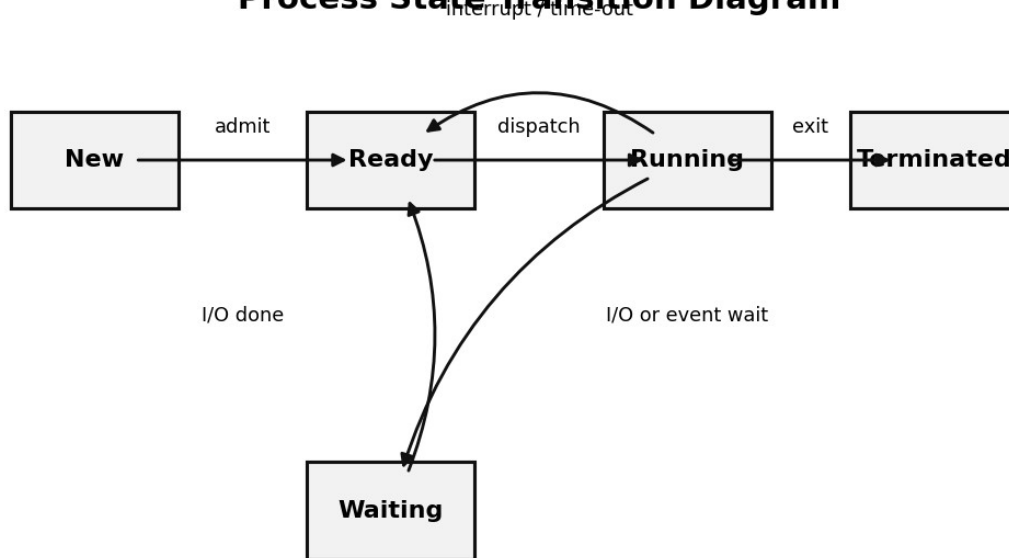
Analogy: a **process is a house; threads are the family members** inside sharing the same kitchen and living room (shared memory) but each with their own bed (own stack).

	Process	Thread
Memory	Separate, isolated	Shared within the process
Creation cost	Heavy (new memory space)	Light
Communication	Inter-Process Comm. (pipes, sockets)	Shared memory (fast, easy)
If one crashes	Others unaffected	Can crash the whole process
Example	Chrome browser	Each browser tab running in parallel

Multithreading = one process running several threads at once (e.g. a bank server handling many customers' requests simultaneously) — this improves responsiveness and uses multi-core CPUs well.

Process states (draw this — common sub-question)

Process State Transition Diagram



Process state transition diagram

- **New:** being created.
- **Ready:** waiting for the CPU (in the ready queue).
- **Running:** currently using the CPU.
- **Waiting/Blocked:** waiting for I/O or an event.
- **Terminated:** finished.

The OS stores each process's details in a **PCB (Process Control Block)** — PID, state, program counter, registers, memory limits. Switching the CPU from one process to another means saving one PCB and loading another; this is a **context switch** (pure overhead — no useful work done during it).

2.2 CPU Scheduling (guaranteed math — master the Gantt chart)

Why: many processes want the CPU but only one can run at a time. The **scheduler** decides the order. Your goal in the exam: draw the **Gantt chart**, then compute averages.

Key formulas (memorize):

- **Completion Time (CT):** the clock time a process finishes.
- **Turnaround Time (TAT) = CT – Arrival Time.**
- **Waiting Time (WT) = TAT – Burst Time.**
- **Response Time:** first time it gets the CPU – Arrival Time.

The algorithms:

- **FCFS (First Come First Served):** run in arrival order. Simple, but a long job delays everyone (the **convoy effect**).
- **SJF (Shortest Job First):** run the smallest burst next. Gives the **minimum average waiting time**, but long jobs may **starve**. (Non-preemptive.)
- **SRTF (Shortest Remaining Time First):** the preemptive version of SJF.
- **Priority scheduling:** highest priority first; low-priority jobs may starve — fixed by **aging** (slowly raising a waiting job's priority).
- **Round Robin (RR):** each process gets a fixed **time quantum**, then goes to the back of the queue. Fair; best for **time-sharing** systems.

Worked example 1 — FCFS vs SJF (all arrive at $t = 0$)

Processes P1, P2, P3 with bursts 24, 3, 3.

FCFS (order P1, P2, P3):

Gantt: | P1 0-24 | P2 24-27 | P3 27-30 |
 WT : P1 = 0, P2 = 24, P3 = 27 $\rightarrow$ Avg WT = $(0+24+27)/3 = 17$ ms
 TAT: P1 = 24, P2 = 27, P3 = 30 $\rightarrow$ Avg TAT = 27 ms

SJF (shortest first — P2, P3, P1):

Gantt: | P2 0-3 | P3 3-6 | P1 6-30 |
 WT : P2 = 0, P3 = 3, P1 = 6 $\rightarrow$ Avg WT = 3 ms (far better than 17!)

This proves **SJF minimises average waiting time**.

Worked example 2 — full four-process problem

Four processes, all arrive at $t = 0$: P1 = 6, P2 = 8, P3 = 7, P4 = 3.

SJF order (by burst): P4(3), P1(6), P3(7), P2(8):

Gantt: | P4 0-3 | P1 3-9 | P3 9-16 | P2 16-24 |
 CT : P4=3, P1=9, P3=16, P2=24
 TAT: 3, 9, 16, 24 $\rightarrow$ Avg TAT = $52/4 = 13$ ms
 WT : 0, 3, 9, 16 $\rightarrow$ Avg WT = $28/4 = 7$ ms

For comparison, **FCFS** in order P1,P2,P3,P4 gives Avg WT = $(0+6+14+21)/4 = 10.25$ ms — SJF wins.

Worked example 3 — Round Robin (quantum = 4)

P1 = 5, P2 = 4, P3 = 2 (all at $t = 0$), quantum = 4:

Gantt: | P1 0-4 | P2 4-8 | P3 8-10 | P1 10-11 |
 CT : P2=8, P3=10, P1=11
 TAT: P1=11, P2=8, P3=10 $\rightarrow$ Avg TAT = $29/3 = 9.67$ ms
 WT : P1=6, P2=4, P3=8 $\rightarrow$ Avg WT = $18/3 = 6$ ms

★ **Predicted question (10 marks):** “Given processes with arrival and burst times, draw the Gantt chart and compute average waiting & turnaround time for FCFS and SJF.” – Draw both Gantt charts, tabulate CT, $TAT = CT - AT$, $WT = TAT - BT$, then average. Always show the table.

2.3 Deadlock, Livelock & Starvation

Deadlock is when a set of processes are stuck forever, each holding a resource the next one needs — a **circular wait** that never resolves. *Analogy:* two people in a narrow corridor, each refusing to step back; nobody moves.

Four Coffman conditions — ALL must hold for deadlock (memorize):

1. **Mutual Exclusion** — a resource can be used by only one process at a time.
2. **Hold and Wait** — a process holds one resource while waiting for another.
3. **No Preemption** — a resource can't be forcibly taken; it must be released voluntarily.
4. **Circular Wait** — a closed chain: P1 waits for P2, P2 waits for P3, ... Pn waits for P1.

Handling deadlock:

- **Prevention:** break any one of the four conditions.
- **Avoidance:** grant resources only if the system stays *safe* — **Banker's Algorithm**.
- **Detection & Recovery:** allow deadlock, detect it, then kill/rollback a process.
- **Ignore (Ostrich):** pretend it can't happen (used by many OSes since deadlocks are rare).

Livelock (your list wrote “Lovelock” — the correct term is **Livelock**): processes are **not blocked** but keep **reacting to each other and changing state**, making **no progress** — like two people in a corridor who keep stepping the *same* way to let each other pass, forever mirroring.

Starvation: a process **waits indefinitely** because others keep being chosen ahead of it (e.g. low priority under Priority scheduling). Fixed by **aging**.

2.4 Banker's Algorithm (deadlock avoidance)

The idea. The OS behaves like a cautious **banker** giving loans (resources). It grants a request **only if**, after granting, it can still let **every** process finish in some order — i.e. the system stays in a **safe state**. A **safe state** = there exists a **safe sequence** where each process can get what it needs, finish, and release.

Matrices: Allocation (held now), **Max** (maximum it may need), **Need = Max – Allocation**, **Available** (free resources).

Worked example. 3 resource types A, B, C. Total = (10, 5, 7). Available = (3, 3, 2).

Process	Allocation	Max	Need = Max – Alloc
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2

Process	Allocation	Max	Need = Max – Alloc
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

Find a safe sequence (pick any process whose **Need** $\leq$ **Available**, run it, add its Allocation back to Available):

Available = (3,3,2)

P1: Need(1,2,2) $\leq$ (3,3,2) -> run -> Available = (3,3,2)+(2,0,0) = (5,3,2)

P3: Need(0,1,1) $\leq$ (5,3,2) -> run -> Available = (5,3,2)+(2,1,1) = (7,4,3)

P4: Need(4,3,1) $\leq$ (7,4,3) -> run -> Available = (7,4,3)+(0,0,2) = (7,4,5)

P0: Need(7,4,3) $\leq$ (7,4,5) -> run -> Available = (7,4,5)+(0,1,0) = (7,5,5)

P2: Need(6,0,0) $\leq$ (7,5,5) -> run -> done

Safe sequence = <P1, P3, P4, P0, P2> – the state is SAFE, so the request can be granted.

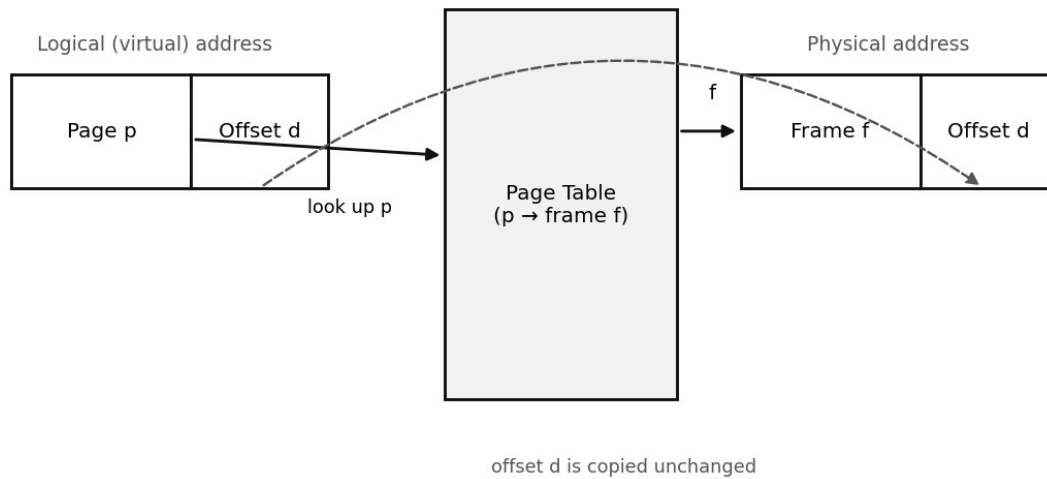
2.5 Virtual Memory, Paging & Segmentation

Virtual memory lets a program believe it has a huge, private, continuous memory even though physical RAM is small. Only the **actively used parts** stay in RAM; the rest sit on disk (the **swap** area) and are brought in when needed. This lets you run programs bigger than RAM and run many programs at once.

Paging

Paging splits **logical memory into fixed-size pages** and **physical memory into equal-size frames**. A **page table** maps each page to a frame. Because sizes are equal, there is **no external fragmentation** (only small **internal** fragmentation in the last page).

Paging: Logical → Physical Address Translation



Paging: logical to physical address translation

A logical address = (**page number p**, **offset d**). The page table converts **p** → **frame f**; the **offset d** is copied unchanged. Physical address = (frame f, offset d).

Segmentation

Segmentation divides memory by **logical units of variable size** — code segment, data segment, stack segment — matching how a programmer thinks. It can cause **external fragmentation** (gaps between segments).

	Paging	Segmentation
Division	Fixed-size pages	Variable-size logical segments
Fragmentation	Internal	External
Viewpoint	Physical (hardware)	Logical (programmer)

Page faults, thrashing, replacement

- **Page fault:** the needed page isn't in RAM → the OS fetches it from disk (slow).
- **Thrashing:** so many page faults that the CPU spends nearly all its time swapping instead of working.
- **Page-replacement algorithms:** **FIFO** (oldest out), **LRU** (Least Recently Used out), **Optimal** (the one not needed for the longest — theoretical best). **FIFO** can suffer **Belady's anomaly** (more frames — more faults).

Worked LRU example — reference string 7 0 1 2 0 3 0 4, 3 frames:

```

7 -> fault [7]
0 -> fault [7,0]
1 -> fault [7,0,1]

```

```

2 -> fault, evict 7 (LRU) [0,1,2]... wait order: [2,0,1]
0 -> hit
3 -> fault, evict 1 -> [2,0,3]
0 -> hit
4 -> fault, evict 2 -> [4,0,3]
Total page faults = 6

```

2.6 Hard Disk Math

Disk access time = Seek time + Rotational latency + Transfer time.

- **Seek time:** move the head to the right track.
- **Rotational latency (average):** wait for the sector to spin under the head = time for **half** a rotation. avg latency = $(60 / \text{RPM}) / 2$ seconds. For 7200 RPM: one rotation = $60/7200 = 8.33 \text{ ms}$ → **avg latency $\approx 4.17 \text{ ms}$.**
- **Transfer time:** data size $\div$ transfer rate.

Disk-scheduling algorithms (reduce total head movement)

FCFS, SSTF (Shortest Seek Time First), **SCAN** (elevator — sweep to one end then back), **C-SCAN, LOOK, C-LOOK**.

★ **Predicted question (10 marks):** “A disk has tracks 0–199, head at 53. Requests: 98, 183, 37, 122, 14, 124, 65, 67. Compute total head movement for FCFS and SSTF.”

FCFS (service in request order):

```

53->98->183->37->122->14->124->65->67
moves: 45 +85 +146 +85 +108 +110 +59 +2 = 640 tracks

```

SSTF (always go to the nearest pending request):

```

53->65->67->37->14->98->122->124->183
moves: 12 +2 +30 +23 +84 +24 +2 +59 = 236 tracks    (much better)

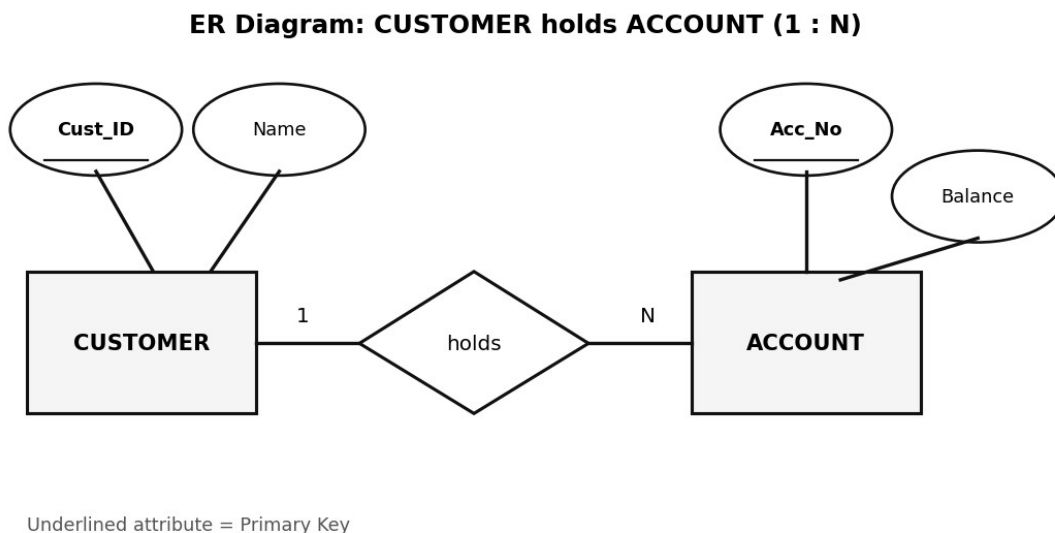
```

3. Database Management

What a DBMS is. A **Database Management System** (MySQL, Oracle, PostgreSQL, SQL Server) stores, organises, secures and retrieves data efficiently, and lets many users share it safely. A bank's **Core Banking System** is essentially a huge, carefully-designed relational database.

3.1 ER Diagrams

The idea. An **Entity-Relationship (ER) diagram** is the *blueprint* you draw **before** creating tables. It shows the real-world things (entities), their properties (attributes), and how they relate.



ER diagram: CUSTOMER holds ACCOUNT (1:N)

Notation:

- **Rectangle = Entity** (a thing: CUSTOMER, ACCOUNT, LOAN).
- **Ellipse = Attribute** (a property: Name, Balance). An **underlined** attribute is the **primary key**.
- **Diamond = Relationship** (a verb linking entities: *holds*, *takes*).
- **Lines** connect them; **1**, **N**, **M** show **cardinality**.

Cardinality (how many relate to how many):

- **1 : 1** — one person has one passport.
- **1 : N** — one customer holds many accounts (most common in banking).
- **M : N** — students ↔ courses. When you convert an **M:N** relationship to tables, you must create a **junction/bridge table** in the middle.

Keys (the hierarchy): *Super key* → *Candidate key* → **Primary key** (the chosen candidate; unique + never null). A **Foreign key** in one table points to the primary key of another table and enforces **referential integrity** (you can't have an account whose customer doesn't exist).

3.2 Normalization / Denormalization

The idea. Normalization re-organises tables to **remove duplicate data** and prevent three **anomalies**:

- **Insert anomaly:** can't add data because other data is missing.
- **Update anomaly:** must change the same fact in many rows (risk of inconsistency).
- **Delete anomaly:** deleting a row accidentally loses another fact.

You normalise step by step:

- **1NF (First Normal Form):** every cell holds a **single atomic value** — no repeating groups, no comma-separated lists.
- **2NF:** it is in 1NF **and** has **no partial dependency** — no non-key column depends on only *part* of a composite primary key.
- **3NF:** it is in 2NF **and** has **no transitive dependency** — a non-key column must not depend on another non-key column.
- **BCNF:** a stricter 3NF — every determinant must be a candidate key.

Memory hook: “Each non-key column depends on **the key, the whole key, and nothing but the key** — so help me Codd.” (1NF = the key; 2NF = the whole key; 3NF = nothing but the key.)

Worked example. Un-normalized: Student(Roll, Name, Course1, Course2, Dept, Dept_Head).

- **1NF:** remove repeating Course1/Course2 → make each course a separate row: (Roll, Name, Course, Dept, Dept_Head).
- **2NF:** Name, Dept depend only on Roll (part of the key {Roll, Course}) → split out Student(Roll, Name, Dept) and Enrollment(Roll, Course).
- **3NF:** Dept_Head depends on Dept (a non-key column) — transitive — split out Department(Dept, Dept_Head).

Final tables: Student(Roll, Name, Dept), Department(Dept, Dept_Head), Enrollment(Roll, Course).

Denormalization deliberately **adds some redundancy back** (merging tables, storing pre-computed totals) to make **reads faster** — used in reporting dashboards and data warehouses. Trade-off: faster reads, but more storage and harder updates. Banks denormalize reporting databases while keeping the transactional core fully normalized.

3.3 SQL Queries (write these fluently)

Sample schema: Account(acc_no, name, branch, balance) and Txn(txn_id, acc_no, amount, type, ts).

```
-- 1) Filter and sort
SELECT name, balance FROM Account
WHERE balance > 100000
ORDER BY balance DESC;

-- 2) Aggregate + GROUP BY + HAVING
SELECT branch, COUNT(*) AS num_accounts, SUM(balance) AS total
FROM Account
GROUP BY branch
HAVING SUM(balance) > 5000000;

-- 3) INNER JOIN two tables
SELECT a.name, t.amount, t.type
FROM Account a
JOIN Txn t ON a.acc_no = t.acc_no
WHERE t.type = 'DEBIT';

-- 4) Second-highest balance (classic exam question)
SELECT MAX(balance) FROM Account
WHERE balance < (SELECT MAX(balance) FROM Account);

-- 5) Create a table with a primary key
CREATE TABLE Account (
    acc_no    INT PRIMARY KEY,
    name      VARCHAR(50) NOT NULL,
    branch    VARCHAR(30),
    balance   DECIMAL(15,2) DEFAULT 0
);

-- 6) Update inside a transaction (fund transfer)
BEGIN;
UPDATE Account SET balance = balance - 1000 WHERE acc_no = 101;
UPDATE Account SET balance = balance + 1000 WHERE acc_no = 202;
COMMIT;
```

Know the SQL command families:

- **DDL** (Data Definition): CREATE, ALTER, DROP, TRUNCATE.
- **DML** (Data Manipulation): SELECT, INSERT, UPDATE, DELETE.
- **DCL** (Data Control): GRANT, REVOKE.
- **TCL** (Transaction Control): COMMIT, ROLLBACK, SAVEPOINT.

Joins: INNER (matching rows only), LEFT (all left + matches), RIGHT, FULL OUTER, CROSS, SELF.

3.4 Indexing

The idea. An **index** is an extra data structure (usually a **B-tree** / **B+ tree**) that lets the database **find rows fast** instead of scanning the whole table — exactly like the index at the back of a textbook.

- **Clustered index:** the table's rows are physically **sorted** in index order — only **one** per table (usually the primary key).
- **Non-clustered index:** a separate structure holding the key + a pointer to the row — you can have **many**.
- **Trade-off:** indexes make **reads (SELECT)** much faster but **writes (INSERT/UPDATE/DELETE)** a little slower (the index must be maintained), and they use extra storage.

Banking angle: in a Core Banking System you index `acc_no`, `card_no`, and `txn_ts` so balance checks and statement queries return instantly across millions of rows.

3.5 Stored Procedures & Triggers

- A **Stored Procedure** is a **named block of SQL** stored in the database and called by name (optionally with parameters). Benefits: reusable, precompiled (faster), reduces network traffic, improves security (users call the procedure without touching tables directly).
- A **Trigger** is a special procedure that **fires automatically** when an event happens — **BEFORE / AFTER** an **INSERT / UPDATE / DELETE**. Used for **audit logging**, enforcing business rules, and auto-updating values.

```
-- Trigger: log every large transaction for audit
CREATE TRIGGER log_big_txn
AFTER INSERT ON Txn
FOR EACH ROW
WHEN (NEW.amount > 10000000)
BEGIN
    INSERT INTO AuditLog(acc_no, amount, ts)
    VALUES (NEW.acc_no, NEW.amount, NOW());
END;
```

3.6 ACID Properties (almost certain — banks love this)

The idea. A **transaction** is one logical unit of work (e.g. a fund transfer). **ACID** are the four guarantees that keep the database correct even during crashes and concurrency.

- **A — Atomicity:** *all or nothing*. A transfer must **debit AND credit**, or do neither. If the debit succeeds but the credit fails, everything **rolls back**.
- **C — Consistency:** the database goes from one **valid state to another**; all rules/constraints stay satisfied (total money is conserved).
- **I — Isolation:** concurrent transactions don't interfere; the result is as if they ran **one after another**. Prevents *dirty reads, non-repeatable reads, phantom reads*.
- **D — Durability:** once a transaction is **committed**, its effect **survives** power failure or crash (written to non-volatile storage / logs).

Banking example (transfer 1000 A — B): *Atomicity* — both legs or none; *Consistency* — the total across A and B is unchanged; *Isolation* — a simultaneous balance enquiry never sees a half-done transfer; *Durability* — after “Success”, the transfer stays even if the server crashes a second later.

3.7 RAID (Redundant Array of Independent Disks)

The idea. RAID combines several physical disks into one logical unit for **speed, fault tolerance**, or both.

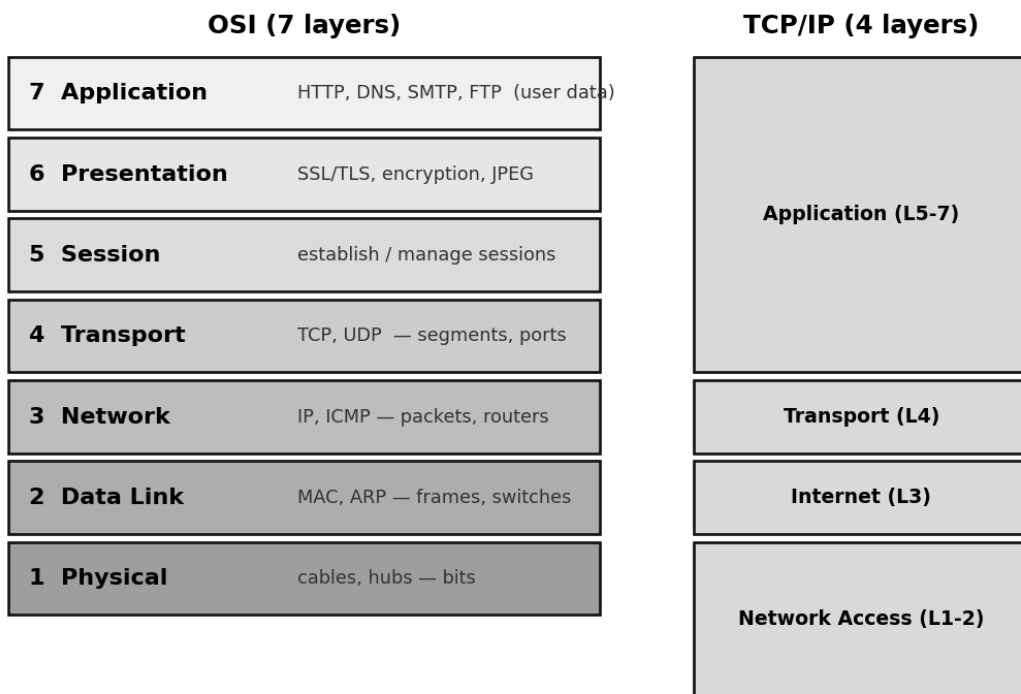
Level	Technique	Min disks	Survives	Note
RAID 0	Striping (split data across disks)	2	no failure	Fast, no redundancy
RAID 1	Mirroring (exact copy)	2	1 disk	50% usable capacity
RAID 5	Striping + distributed parity	3	1 disk	Popular balance
RAID 6	Striping + double parity	4	2 disks	Extra safety
RAID 10	Mirror then stripe (1+0)	4	1 per mirror	Fast + redundant

Memory hook: 0 = zero safety (pure speed), 1 = one mirror copy, 5 = parity spread out, 10 = mirror + stripe (best of both). Banks typically run databases on **RAID 5, 6 or 10**.

4. Networking & Security

4.1 OSI Model (7 layers — draw and map it)

The idea. The **OSI (Open Systems Interconnection)** model breaks networking into **7 layers**, each with one job. At the sender, data moves **down** the stack (each layer adds a header — *encapsulation*); at the receiver it moves **up** (each layer removes its header).



Mnemonic: "All People Seem To Need Data Processing"

OSI 7 layers vs TCP/IP

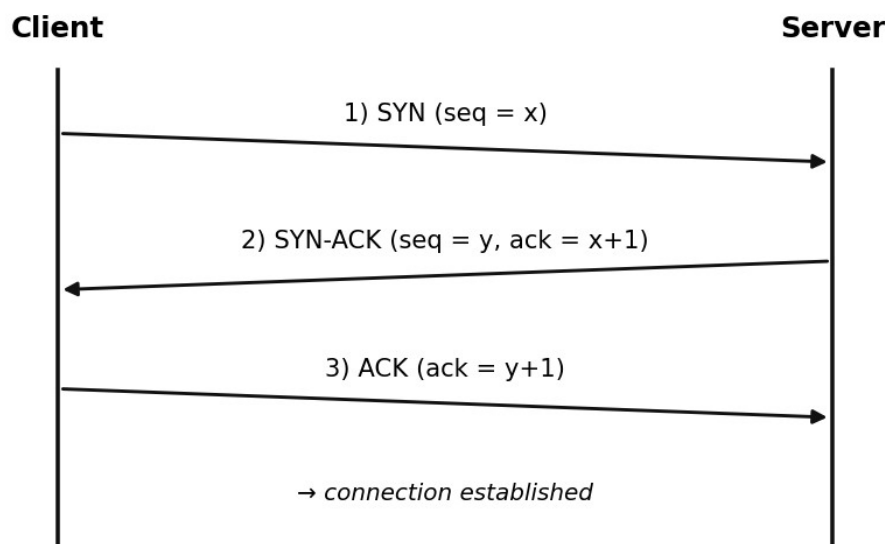
Mnemonic (top → bottom): “All People Seem To Need Data Processing.”

#	Layer	Job	Devices / Protocols	Data unit
7	Application	Services the user sees	HTTP, DNS, SMTP, FTP	Data
6	Presentation	Encrypt / compress / format	SSL/TLS, JPEG	Data
5	Session	Start & manage sessions	RPC, NetBIOS	Data
4	Transport	Reliable delivery, ports	TCP, UDP	Segment
3	Network	Logical	IP, ICMP,	Packet

#	Layer	Job	Devices / Protocols	Data unit
		addressing, routing	Router	
2	Data Link	MAC addressing, framing	Switch , ARP	Frame
1	Physical	Bits on the medium	cables, hubs	Bits

TCP vs UDP. **TCP** = *connection-oriented, reliable, ordered* (uses the 3-way handshake; used by web, email, file transfer). **UDP** = *connectionless, fast, no delivery guarantee* (used by DNS, live video, VoIP, online games) — speed over reliability.

TCP 3-Way Handshake



TCP 3-way handshake

The **3-way handshake** sets up a TCP connection: **SYN** → **SYN-ACK** → **ACK**. Connection close uses a 4-way **FIN/ACK** exchange.

4.2 Error Detection & Correction Math

Data can get corrupted in transit; these methods catch (and sometimes fix) it.

- **Parity bit:** add one extra bit so the total number of 1s is **even** (even parity) or **odd**. Detects a single-bit error only.
- **Checksum:** add up the data words; send the sum. The receiver re-adds — a mismatch means an error.
- **CRC (Cyclic Redundancy Check):** treat the data as a big binary number, **divide it by a generator polynomial** using XOR (mod-2) division, and append the **remainder** as

CRC bits. The receiver divides again; **remainder 0 = no error**. Strong; used in Ethernet, Wi-Fi.

- **Hamming code:** places parity bits at positions 1, 2, 4, 8 ... to **detect and correct** a single-bit error. For m data bits you need r parity bits where $2^r \geq m + r + 1$.

Worked CRC example (10 marks). Data = 1101011011, generator $G = 10011$ (degree 4).

- 1) Append 4 zeros (degree of G): 11010110110000
- 2) XOR-divide by 10011 step by step (mod-2 division)
- 3) The remainder = 1110
- 4) Transmitted frame = data + CRC = 1101011011 1110

The receiver divides the received frame by 10011; if the remainder is 0, the data is accepted.

4.3 Protocols (one-liners = full marks)

Protocol	Full form / role	Port
DNS	Domain Name System — resolves a name (google.com) to an IP	53
DHCP	Dynamic Host Configuration — auto-assigns IP addresses	67/68
ARP	Address Resolution — finds the MAC for a given IP	—
NAT	Network Address Translation — many private IPs share one public IP	—
HTTP / HTTPS	Web pages / secure web (encrypted with TLS)	80 / 443
SSL / TLS	Encrypts data in transit; TLS is the modern replacement for SSL	—
SMTP	Send email	25
POP3	Download email (removes it from the server)	110
IMAP	Sync email (keeps it on the server; multi-device)	143
FTP	File transfer	20/21

Memory hook: SMTP = Send Mail To People. POP = Pull Off (download + remove). IMAP = keeps mail In (on server).

4.4 Network Devices

- **Hub (Layer 1):** dumb repeater — copies incoming signal to *all* ports (obsolete, causes collisions).

- **Switch (Layer 2):** learns **MAC addresses** and forwards a frame only to the correct port — creating separate collision domains, far more efficient than a hub.
- **Router (Layer 3):** connects **different networks**, forwards **packets** using **IP addresses** and a routing table; also does NAT.
- **Bridge:** joins two LAN segments. **Gateway:** translates between different protocols/networks. **Modem:** converts digital $\leftrightarrow$ analog. **Firewall:** filters traffic by security rules.

Quick: *Switch = smart, within one network (MAC). Router = connects networks (IP).*

4.5 Subnetting Math (almost guaranteed — practise!)

The idea. An IPv4 address is **32 bits** = four octets (e.g. 192.168.10.5). A **subnet mask** marks which bits are the **network** part (1s) and which are the **host** part (0s). **CIDR /n** means the first n bits are network. **Subnetting** splits one network into several smaller ones.

Key formulas:

- **Number of subnets** = $2^{\text{(borrowed bits)}}$.
- **Usable hosts per subnet** = $2^{\text{(host bits)}} - 2$ (subtract the network address and the broadcast address).
- **Block size** = $256 - (\text{value of the subnet-mask octet})$.

Worked example (the classic bank question). “Given 192.168.10.0/24, create 4 equal subnets. Find the new mask, hosts per subnet, and each subnet’s network / first / last / broadcast address.”

To make **4 subnets** from a /24, **borrow 2 bits** ($2^2 = 4$). New prefix = **/26**. Mask = **255.255.255.192**. Host bits = 6, so usable hosts = $2^6 - 2 = 62$. Block size = $256 - 192 = 64$.

Subnet	Network	First usable	Last usable	Broadcast
1	192.168.10.0	.1	.62	.63
2	192.168.10.64	.65	.126	.127
3	192.168.10.128	.129	.190	.191
4	192.168.10.192	.193	.254	.255

Must-memorize facts: Private ranges 10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16. Loopback 127.0.0.1. Classes: A (1–126), B (128–191), C (192–223). IPv6 is **128-bit**.

4.6 Bandwidth & Multiplexing Math

- **Bandwidth** = the maximum data rate of a link (bits/second). **Throughput** = the rate actually achieved.
- **Nyquist (noiseless channel):** $C = 2 \cdot B \cdot \log_2(L)$ bps — B = bandwidth (Hz), L = number of signal levels.
- **Shannon (noisy channel):** $C = B \cdot \log_2(1 + S/N)$ bps. If SNR is given in dB, first convert: $S/N = 10^{(\text{dB}/10)}$.

- **Transfer time = file size ÷ bandwidth.** *Example:* a 10 MB file over an 8 Mbps link = $(10 \times 8 \text{ Mb}) / 8 \text{ Mbps} = 10 \text{ s}$.

Multiplexing shares one physical medium among many signals:

- **FDM (Frequency Division):** each signal gets its own frequency band (radio, cable TV).
- **TDM (Time Division):** each signal gets its own time slot (digital telephony). *Statistical TDM* gives slots on demand.
- **WDM:** different light wavelengths on one optical fibre.

Worked Shannon example (10 marks): “A channel of 3000 Hz bandwidth has SNR = 30 dB. Find the maximum data rate.” — $S/N = 10^{(30/10)} = 1000$; $C = 3000 \cdot \log_2(1001) \approx 3000 \cdot 9.97 \approx 29,900 \text{ bps} \approx 30 \text{ kbps}$.

4.7 Security — the CIA Triad

The foundation of information security — the three properties you must protect:

- **Confidentiality:** only authorised people can read the data (encryption, access control, passwords).
- **Integrity:** data is not altered without authorisation (hashing, checksums, digital signatures).
- **Availability:** systems and data are accessible when needed (backups, redundancy, DDoS protection).

4.8 Hashing vs Encryption

- **Hashing is one-way** (irreversible): any input — a **fixed-length digest** (MD5, SHA-256). The same input always gives the same hash; you **cannot** get the original back. Used for **password storage** (with a *salt*) and **integrity** checks.
- **Encryption is two-way** (reversible with a key). Used for **confidentiality** — protecting data so only the key-holder can read it.

Quick: Hash = verify (can't reverse). Encrypt = protect (reversible with key).

4.9 Symmetric vs Asymmetric Cryptography (very common 10-marker)

Symmetric encryption



AES, DES, 3DES — fast; problem: sharing the secret key

Asymmetric encryption



RSA, ECC — slower; solves key sharing, enables digital signatures

Symmetric vs asymmetric encryption

	Symmetric	Asymmetric
Keys	One shared secret key	Key pair: public + private
Speed	Fast	Slow
Algorithms	AES, DES, 3DES, Blowfish	RSA, ECC, Diffie-Hellman
Best for	Bulk data encryption	Key exchange, digital signatures
Weakness	Sharing the secret key safely	Computationally heavy

How they team up in HTTPS/TLS: asymmetric crypto is used **once** to securely exchange a symmetric *session key*; then fast symmetric encryption protects the actual data. Best of both worlds.

Digital signature: the sender encrypts a **hash of the message** with their **private** key. Anyone can verify it with the sender's **public** key — proving **authenticity + integrity + non-repudiation** (the sender can't deny it).

4.10 Cyber Attacks (know the definitions)

- **Phishing:** fake emails/websites trick users into revealing credentials. **Spear phishing** = targeted at a specific person.

- **Malware:** **virus** (attaches to a file), **worm** (self-spreading), **trojan** (disguised), **ransomware** (encrypts files, demands payment), **spyware**.
- **DoS / DDoS:** flood a server so it becomes unavailable (attacks **Availability**).
- **Man-in-the-Middle (MITM):** the attacker secretly sits between two parties, reading/altering messages.
- **SQL Injection:** malicious SQL typed into an input field to read/alter the database — prevented with **parameterised queries** / **prepared statements**.
- **XSS (Cross-Site Scripting):** inject a script into a web page viewed by others.
- **Brute force:** try every password. **Zero-day:** exploit an unknown, unpatched flaw.

4.11 Defence Tools — Firewall, IDS, IPS, VPN

- **Firewall:** filters traffic against rules (allow/deny by IP, port, protocol) — the network's gatekeeper. Can be hardware or software.
- **IDS (Intrusion Detection System):** **detects and alerts** on suspicious activity — **passive**, like a **burglar alarm**.
- **IPS (Intrusion Prevention System):** detects **and actively blocks** the threat — **inline**, like a **security guard** who stops the intruder.
- **VPN (Virtual Private Network):** builds an **encrypted tunnel** across the public internet for secure remote access; also hides your IP.

Memory hook: IDS = Detect (alarm). IPS = Prevent (block).

4.12 More Security Terms (each worth a line)

- **Steganography:** **hiding** a secret message *inside* an ordinary file (e.g. inside an image) so its very existence is hidden — different from cryptography, which scrambles a message but doesn't hide that one exists.
- **SIM Cloning:** copying a SIM's identity (IMSI + secret key Ki) so the attacker can receive the victim's calls/SMS — a threat to **SMS-based OTP**.
- **OTP (One-Time Password):** a code valid for a single login or short time window. **TOTP** = time-based (Google Authenticator); **HOTP** = counter-based. Core to **2FA/MFA**. SMS-OTP is vulnerable to SIM swap/cloning, so **app-based TOTP is safer**.
- **2FA / MFA:** combine factors — *something you know* (password) + *something you have* (phone/token) + *something you are* (fingerprint/face).
- **Encryption at rest vs in transit:** *at rest* protects stored data (disk/DB encryption); *in transit* protects data moving over the network (TLS).

5. Programming

5.1 Core Concepts (explain these clearly)

Conditional statements let a program choose a path.

- `if / else if / else` — choose based on conditions.
- `switch` — multi-way branch on a single value.

Loops repeat a block.

- `for` — when you know the count.
- `while` — repeat while a condition is true (checked first).
- `do...while` — runs the body at least once (condition checked after).

break vs continue (favourite viva question):

- **break** — leave the loop **completely**, right now.
- **continue** — skip the **rest of the current turn** and jump to the next iteration.

```
for (int i = 1; i <= 6; i++) {  
    if (i == 3) continue;    // skip printing 3  
    if (i == 5) break;       // stop the loop before 5  
    printf("%d ", i);        // Output: 1 2 4  
}
```

Arrays store many values of the same type in **contiguous memory**, indexed 0 ... n-1. `int a[5];` → `a[0]` is first. A 2-D array `a[i][j]` is a matrix.

Pointers hold a **memory address**. `&x` = “address of x”; `*p` = “the value stored at the address in p” (dereference). Pointers enable dynamic memory (`malloc/free`), pass-by-reference, linked lists and trees.

```
int x = 10;  
int *p = &x;           // p holds the address of x  
printf("%d\n", *p);    // prints 10 (value at that address)  
*p = 20;                // changes x to 20 through the pointer
```

Recursion — a function that **calls itself** on a smaller input until a **base case** stops it. Every recursion needs a base case, or it runs forever (stack overflow).

```
int fact(int n) {  
    if (n <= 1) return 1;    // base case  
    return n * fact(n - 1);  // recursive step  
}
```

5.2 Complete Programs (full code — the way the exam wants them)

Each program below is **complete and runnable** — with headers, `main()`, input/output, and a sample run. Learn the **logic**, then you can rewrite from memory.

Program 1 — Check whether a number is prime

```
#include <stdio.h>
int main() {
    int n, i, isPrime = 1;
    printf("Enter a number: ");
    scanf("%d", &n);

    if (n < 2) isPrime = 0;
    for (i = 2; i * i <= n; i++) {
        if (n % i == 0) { isPrime = 0; break; }
    }

    if (isPrime) printf("%d is a prime number\n", n);
    else        printf("%d is NOT a prime number\n", n);
    return 0;
}
/* Sample run:  Enter a number: 17  ->  17 is a prime number */
```

Program 2 — Factorial of a number

```
#include <stdio.h>
int main() {
    int n, i;
    long long fact = 1;
    printf("Enter a number: ");
    scanf("%d", &n);

    for (i = 2; i <= n; i++)
        fact = fact * i;

    printf("Factorial of %d = %lld\n", n, fact);
    return 0;
}
/* Sample run:  Enter a number: 5  ->  Factorial of 5 = 120 */
```

Program 3 — Fibonacci series (first n terms)

```
#include <stdio.h>
int main() {
    int n, i, a = 0, b = 1, next;
    printf("How many terms? ");
    scanf("%d", &n);

    printf("Fibonacci: ");
    for (i = 1; i <= n; i++) {
        printf("%d ", a);
        next = a + b;
        a = b;
        b = next;
    }
    printf("\n");
    return 0;
}
```



```

}
/* Sample run:  How many terms? 7  ->  Fibonacci: 0 1 1 2 3 5 8 */

```

Program 4 — Check a palindrome number

```

#include <stdio.h>
int main() {
    int n, original, reversed = 0, digit;
    printf("Enter a number: ");
    scanf("%d", &n);
    original = n;

    while (n > 0) {
        digit = n % 10;
        reversed = reversed * 10 + digit;
        n = n / 10;
    }

    if (original == reversed) printf("%d is a palindrome\n",
original);
    else printf("%d is NOT a palindrome\n",
original);
    return 0;
}
/* Sample run:  Enter a number: 121  ->  121 is a palindrome */

```

Program 5 — Reverse a string

```

#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i;
    printf("Enter a string: ");
    scanf("%s", s);

    printf("Reversed: ");
    for (i = strlen(s) - 1; i >= 0; i--)
        printf("%c", s[i]);
    printf("\n");
    return 0;
}
/* Sample run:  Enter a string: hello  ->  Reversed: olleh */

```

Program 6 — Largest element in an array

```

#include <stdio.h>
int main() {
    int n, i, a[100], max;
    printf("How many numbers? ");
    scanf("%d", &n);
    printf("Enter %d numbers: ", n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
}

```

```

    max = a[0];
    for (i = 1; i < n; i++)
        if (a[i] > max) max = a[i];

    printf("Largest = %d\n", max);
    return 0;
}
/* Sample run:  Enter: 3 9 2 7  ->  Largest = 9 */

```

Program 7 — Sum and average of array elements

```

#include <stdio.h>
int main() {
    int n, i, a[100], sum = 0;
    printf("How many numbers? ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    for (i = 0; i < n; i++) sum += a[i];
    printf("Sum = %d, Average = %.2f\n", sum, (float)sum / n);
    return 0;
}
/* Sample run:  4 numbers: 10 20 30 40  ->  Sum = 100, Average = 25.00
*/

```

Program 8 — Swap two numbers without a third variable

```

#include <stdio.h>
int main() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    a = a + b;
    b = a - b;
    a = a - b;

    printf("After swap: a = %d, b = %d\n", a, b);
    return 0;
}
/* Sample run:  5 8  ->  After swap: a = 8, b = 5 */

```

Program 9 — Count vowels in a string

```

#include <stdio.h>
#include <string.h>
int main() {
    char s[100];
    int i, count = 0;
    printf("Enter a string: ");
    scanf("%s", s);

```

```

    for (i = 0; s[i] != '\0'; i++) {
        char c = s[i];
        if (c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
            c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
            count++;
    }
    printf("Number of vowels = %d\n", count);
    return 0;
}
/* Sample run:  education  ->  Number of vowels = 5 */

```

Program 10 — Armstrong number (e.g. $153 = 1^3 + 5^3 + 3^3$)

```

#include <stdio.h>
int main() {
    int n, original, digit, sum = 0;
    printf("Enter a 3-digit number: ");
    scanf("%d", &n);
    original = n;

    while (n > 0) {
        digit = n % 10;
        sum += digit * digit * digit;
        n = n / 10;
    }

    if (sum == original) printf("%d is an Armstrong number\n",
original);
    else                  printf("%d is NOT an Armstrong number\n",
original);
    return 0;
}
/* Sample run:  153  ->  153 is an Armstrong number */

```

Program 11 — GCD of two numbers (Euclid's algorithm)

```

#include <stdio.h>
int main() {
    int a, b, temp;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);

    while (b != 0) {
        temp = b;
        b = a % b;
        a = temp;
    }
    printf("GCD = %d\n", a);
    return 0;
}
/* Sample run:  36 24  ->  GCD = 12 */

```

Program 12 — Bubble sort (ascending)

```
#include <stdio.h>
int main() {
    int n, i, j, temp, a[100];
    printf("How many numbers? ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);

    for (i = 0; i < n - 1; i++)
        for (j = 0; j < n - 1 - i; j++)
            if (a[j] > a[j + 1]) {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }

    printf("Sorted: ");
    for (i = 0; i < n; i++) printf("%d ", a[i]);
    printf("\n");
    return 0;
}
/* Sample run:  5 1 4 2  ->  Sorted: 1 2 4 5 */
```

Program 13 — Linear search

```
#include <stdio.h>
int main() {
    int n, i, key, a[100], found = -1;
    printf("How many numbers? ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter value to find: ");
    scanf("%d", &key);

    for (i = 0; i < n; i++)
        if (a[i] == key) { found = i; break; }

    if (found != -1) printf("Found at position %d\n", found + 1);
    else
        printf("Not found\n");
    return 0;
}
```

Program 14 — Binary search (sorted array)

```
#include <stdio.h>
int main() {
    int n, i, key, a[100], lo, hi, mid, found = -1;
    printf("How many (sorted) numbers? ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) scanf("%d", &a[i]);
    printf("Enter value to find: ");
    scanf("%d", &key);
```

```

    lo = 0; hi = n - 1;
    while (lo <= hi) {
        mid = (lo + hi) / 2;
        if (a[mid] == key) { found = mid; break; }
        else if (a[mid] < key) lo = mid + 1;
        else hi = mid - 1;
    }

    if (found != -1) printf("Found at position %d\n", found + 1);
    else printf("Not found\n");
    return 0;
}

```

Program 15 — Sum of digits of a number

```

#include <stdio.h>
int main() {
    int n, sum = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    while (n > 0) { sum += n % 10; n /= 10; }
    printf("Sum of digits = %d\n", sum);
    return 0;
}
/* Sample run:  1234  ->  Sum of digits = 10 */

```

Program 16 — Factorial using recursion

```

#include <stdio.h>
long long fact(int n) {
    if (n <= 1) return 1;           // base case
    return n * fact(n - 1);        // recursive call
}
int main() {
    int n;
    printf("Enter a number: ");
    scanf("%d", &n);
    printf("Factorial = %lld\n", fact(n));
    return 0;
}
/* Sample run:  6  ->  Factorial = 720 */

```

Program 17 — Fibonacci using recursion

```

#include <stdio.h>
int fib(int n) {
    if (n < 2) return n;           // base cases: fib(0)=0,
    // fib(1)=1
    return fib(n - 1) + fib(n - 2);
}
int main() {
    int n, i;

```

```

    printf("How many terms? ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) printf("%d ", fib(i));
    printf("\n");
    return 0;
}
/* Sample run:  7  ->  0 1 1 2 3 5 8 */

```

Program 18 — Check leap year

```

#include <stdio.h>
int main() {
    int y;
    printf("Enter a year: ");
    scanf("%d", &y);
    if ((y % 4 == 0 && y % 100 != 0) || (y % 400 == 0))
        printf("%d is a leap year\n", y);
    else
        printf("%d is not a leap year\n", y);
    return 0;
}
/* Sample run:  2024 -> leap year ;  1900 -> not a leap year */

```

Program 19 — Multiplication table of a number

```

#include <stdio.h>
int main() {
    int n, i;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i = 1; i <= 10; i++)
        printf("%d x %d = %d\n", n, i, n * i);
    return 0;
}

```

Program 20 — Matrix addition (2×2)

```

#include <stdio.h>
int main() {
    int a[2][2], b[2][2], c[2][2], i, j;
    printf("Enter matrix A (4 values): ");
    for (i = 0; i < 2; i++) for (j = 0; j < 2; j++) scanf("%d", &a[i]
[j]);
    printf("Enter matrix B (4 values): ");
    for (i = 0; i < 2; i++) for (j = 0; j < 2; j++) scanf("%d", &b[i]
[j]);

    for (i = 0; i < 2; i++)
        for (j = 0; j < 2; j++)
            c[i][j] = a[i][j] + b[i][j];

    printf("Sum matrix:\n");
    for (i = 0; i < 2; i++) {

```

```

        for (j = 0; j < 2; j++) printf("%d ", c[i][j]);
        printf("\n");
    }
    return 0;
}

```

Program 21 — Reverse a number

```

#include <stdio.h>
int main() {
    int n, rev = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    while (n > 0) { rev = rev * 10 + n % 10; n /= 10; }
    printf("Reversed number = %d\n", rev);
    return 0;
}
/* Sample run:  1234  ->  Reversed number = 4321 */

```

Program 22 — Count digits of a number

```

#include <stdio.h>
int main() {
    int n, count = 0;
    printf("Enter a number: ");
    scanf("%d", &n);
    if (n == 0) count = 1;
    while (n > 0) { count++; n /= 10; }
    printf("Number of digits = %d\n", count);
    return 0;
}

```

Exam tip: when asked to “write a program”, also add **one line explaining the logic** and a **sample input/output** — that alone earns 2–3 of the 10 marks. Writing complete code (headers + `main()`) rather than a bare function shows mastery.

5.3 Object-Oriented Programming (OOP)

Class vs Object. A **class** is a **blueprint / template**; an **object** is a real **instance** of that class holding actual values. *Class = the design of a car; object = your specific red Toyota.*

The four pillars (memorize with the banking examples)

1. **Encapsulation** — bundle data + the methods that work on it into one unit, and **hide** the internal data (make fields private, expose public getters/setters). *Example: a BankAccount keeps balance private; you can change it only through `deposit()` and `withdraw()`, which enforce rules.*
2. **Inheritance** — a child class **reuses** a parent’s fields/methods and can add its own. *Example: SavingsAccount and CurrentAccount inherit from Account.* Promotes code reuse. Types: single, multilevel, hierarchical, multiple (via interfaces in Java).
3. **Polymorphism** — “many forms”: the **same interface behaves differently** for different objects. Achieved by **overloading** (compile-time) and **overriding** (run-time).

4. **Abstraction** — show **what** an object does, hide **how** it does it (abstract classes, interfaces). *You press “Withdraw” at an ATM; the internal ledger logic is hidden.*

Overloading vs Overriding (classic 10-marker)

	Overloading	Overriding
Meaning	Same method name, different parameters	Child redefines parent’s method (same signature)
Binding	Compile-time (static)	Run-time (dynamic)
Where	Same class	Between parent and child class
Example	add(int,int) and add(int,int,int)	Animal.sound() overridden by Dog.sound()

// A complete OOP example in Java

```
class Account {                                // parent class
    private double balance = 0;                // encapsulation: private data

    public void deposit(double amt) { balance += amt; }
    public double getBalance() { return balance; }
    public void type() { System.out.println("Generic account"); }
}
```

```
class SavingsAccount extends Account {          // inheritance
    public void type() {                        // overriding (run-
        System.out.println("Savings account");
    }
    // overloading (compile-time): same name, different parameters
    public double interest(double rate)         { return
getBalance()*rate; }
    public double interest(double rate, int years) { return
getBalance()*rate*years; }
}
```

```
public class Main {
    public static void main(String[] args) {
        Account a = new SavingsAccount();      // polymorphism
        a.deposit(1000);
        a.type();                               // prints "Savings
account"
        System.out.println(a.getBalance());    // prints 1000.0
    }
}
```

5.4 MVC Architecture

MVC (Model–View–Controller) splits an application into three parts so that code is easy to maintain and test:

- **Model** — the data and business logic (account balance, database access).
- **View** — what the user sees (the web page / screen).
- **Controller** — receives user input, updates the Model, chooses the View (the middleman).

Flow: *User → Controller → Model (fetch/update data) → View (display) → User*. Benefit: **separation of concerns** (designers work on Views, developers on Models), parallel development, easier testing. Used in Spring MVC, Laravel, ASP.NET MVC — common in banking web applications.

6. Data Structures & Algorithms (DSA)

Why it matters. A **data structure** is a way to organise data so operations are efficient; an **algorithm** is a step-by-step method to solve a problem. Choosing the right one is the difference between a query that takes milliseconds and one that takes minutes.

6.1 Time Complexity (Big-O) — the vocabulary

Big-O describes how the running time grows as input size n grows (worst case).

Notation	Name	Example
$O(1)$	Constant	array access <code>a[i]</code>
$O(\log n)$	Logarithmic	binary search
$O(n)$	Linear	linear search
$O(n \log n)$	Linearithmic	merge sort, quick sort (avg)
$O(n^2)$	Quadratic	bubble/selection/ insertion sort
$O(2^n)$	Exponential	naive recursive Fibonacci

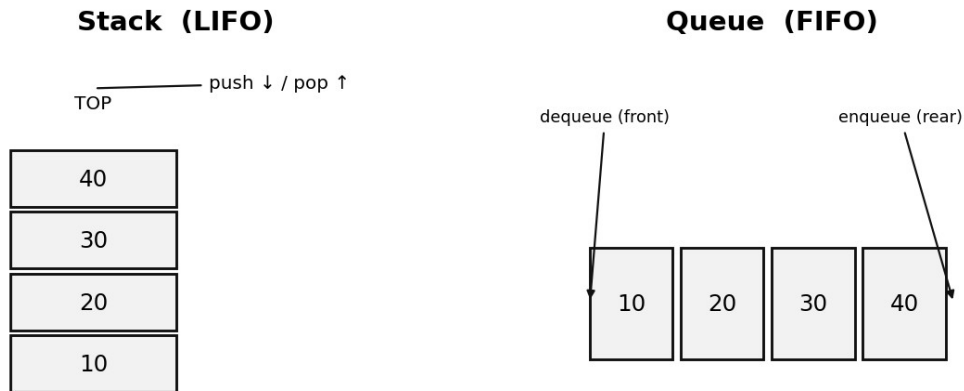
6.2 Linear Data Structures

Array — fixed size, elements in **contiguous memory**. **$O(1)$** random access by index, but inserting/deleting in the middle needs shifting — **$O(n)$** .

Linked List — nodes joined by pointers. **Insert/delete is $O(1)$** (once you're at the node) but there is **no random access** (searching is $O(n)$). Types: **singly** (next only), **doubly** (prev + next), **circular** (last points to first).

Stack (LIFO — Last In, First Out). Operations **push** (add on top), **pop** (remove top), **peek** — all **$O(1)$** . *Analogy: a stack of plates — you take the top one first.* Uses: function-call stack, **undo**, **infix** → **postfix**, expression evaluation, backtracking, DFS.

Queue (FIFO — First In, First Out). **enqueue** at the rear, **dequeue** at the front. *Analogy: a queue at a bank counter — first come, first served.* Uses: CPU scheduling, print queue, **BFS**. Variants: **circular queue**, **priority queue**, **deque** (double-ended).



Stack (LIFO) and Queue (FIFO)

6.3 Infix → Postfix Conversion (favourite question)

Why: computers evaluate **postfix** (Reverse Polish Notation) easily with a stack — no brackets or precedence rules needed at run time.

Operator precedence: ^ (highest) > * / > + - (lowest).

Algorithm (using a stack): scan the infix expression left to right.

1. **Operand** → send straight to the output.
2. **Operator** → while the stack's top operator has **equal or higher** precedence, pop it to output; then push the current operator.
3. **(** → push it. **)** → pop to output until you remove the matching **(**.
4. At the end, pop everything left on the stack.

Worked example — A + B * C - D:

Token	Stack	Output
A		A
+	+	A
B	+	A B
*	+ *	A B
C	+ *	A B C
-	-	A B C * + (pop * then + ; push -)
D	-	A B C * + D
end		A B C * + D -

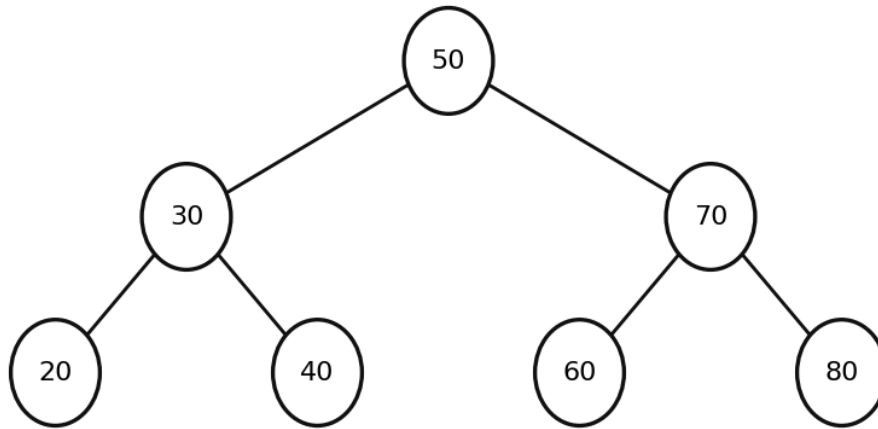
Result: A B C * + D -. Another: (A + B) * (C - D) - A B + C D - *.

6.4 Trees

Tree — a hierarchical structure with one **root**; each node has children. **Binary tree** — each node has at most two children.

Binary Search Tree (BST): for every node, **left subtree < node < right subtree**. This ordering makes search/insert/delete $O(\log n)$ when balanced ($O(n)$ if the tree becomes a skewed chain).

Binary Search Tree (insert 50,30,70,20,40,60,80)



Inorder (L,Root,R) = 20 30 40 50 60 70 80 (sorted)

Preorder = 50 30 20 40 70 60 80 Postorder = 20 40 30 60 80 70 50

Binary Search Tree with traversals

Tree traversals (know all three):

- **Inorder (Left, Root, Right):** on a BST this prints values in **sorted order**.
- **Preorder (Root, Left, Right):** used to **copy** a tree / write an expression tree as prefix.
- **Postorder (Left, Right, Root):** used to **delete** a tree / evaluate an expression tree.

Self-balancing trees: AVL and **Red-Black** trees automatically keep height $\approx \log n$. **B-tree** / **B+ tree** have many children per node (shallow, disk-friendly) — this is exactly what **database indexes** and file systems use.

6.5 Heaps

A **heap** is a **complete binary tree** stored in an array:

- **Max-heap:** every parent $\geq$ its children — the **root is the maximum**.
- **Min-heap:** every parent $\leq$ its children — the **root is the minimum**.

Used for **priority queues** and **Heap Sort** ($O(n \log n)$). Array indexing: for node at index i , children are at $2i+1$ and $2i+2$, parent at $(i-1)/2$.

6.6 Sorting Algorithms (complexity table + one trace)

Algorithm	Best	Average	Worst	Extra space	Stable?
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No

- **Bubble sort:** repeatedly swap adjacent out-of-order pairs; the largest “bubbles” to the end each pass.
- **Selection sort:** repeatedly find the **minimum** of the unsorted part and place it next.
- **Insertion sort:** grow a sorted portion one element at a time (like arranging playing cards in your hand).
- **Merge sort: divide and conquer** — split the array, sort each half, then **merge**. Always $O(n \log n)$ and stable, but needs extra space.
- **Quick sort:** choose a **pivot**, **partition** so smaller elements go left and larger go right, then recurse. Fast in practice; worst case $O(n^2)$ when the pivot is always the smallest/largest.

Bubble sort trace on [5, 1, 4, 2]:

Pass 1: compare & swap adjacent -> [1, 4, 2, 5]

Pass 2: -> [1, 2, 4, 5]

Pass 3: -> [1, 2, 4, 5] (already sorted)

Selection sort trace on [29, 10, 14, 37, 13]:

Find min (10), swap to front -> [10, 29, 14, 37, 13]

Next min (13) -> [10, 13, 14, 37, 29]

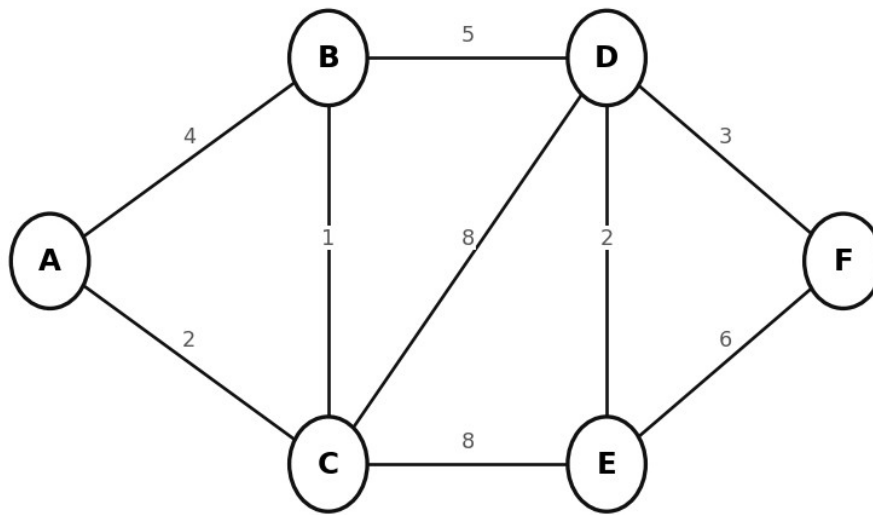
Next min (14) already placed -> [10, 13, 14, 37, 29]

Next min (29) -> [10, 13, 14, 29, 37] sorted

6.7 Graph Traversals — BFS & DFS

A **graph** = **vertices (nodes)** + **edges (links)**. It can be **directed/undirected** and **weighted/unweighted**. Stored as an **adjacency matrix** (grid) or **adjacency list** (list of neighbours).

Weighted graph (for Dijkstra / shortest path)



Weighted graph for traversal / shortest path

- **BFS (Breadth-First Search):** explore **level by level** using a **Queue**. Finds the **shortest path in an unweighted** graph. Time $O(V+E)$.
- **DFS (Depth-First Search):** go **as deep as possible** using a **Stack** (or recursion), then backtrack. Used for cycle detection, topological sort, connected components. Time $O(V+E)$.

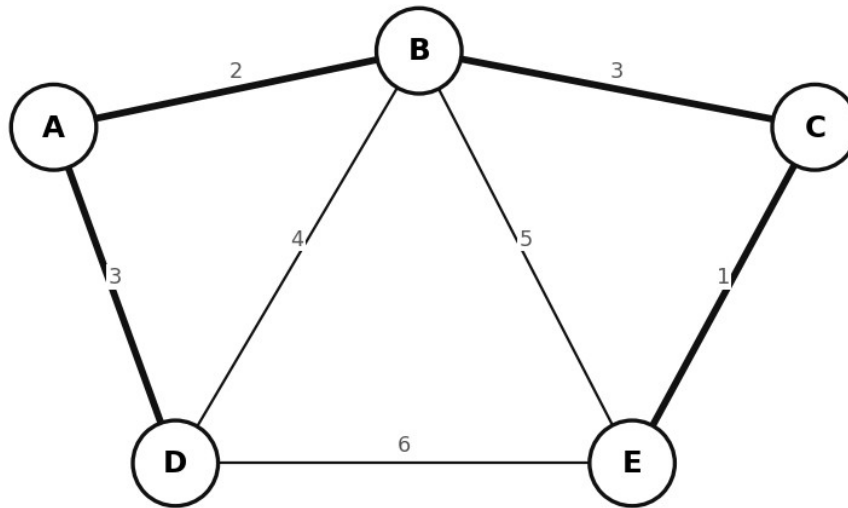
Memory hook: BFS uses a Queue (goes wide); DFS uses a Stack (goes deep).

BFS example from A on the graph above (by increasing distance): A, B, C, D, E, F. **DFS example** from A (go deep first): A, B, C, E, F, D (one possible order).

6.8 Minimum Spanning Tree — Prim's & Kruskal's

A **Minimum Spanning Tree (MST)** connects **all** vertices with the **least total edge weight** and **no cycles** (it has exactly $V - 1$ edges). *Real use: laying cable/road to connect all branches at minimum cost.*

Minimum Spanning Tree (Prim's / Kruskal's)



Bold edges = Minimum Spanning Tree (weight $2+3+3+1 = 9$)

Minimum Spanning Tree (Prim's / Kruskal's)

- **Prim's algorithm:** start from any vertex; repeatedly add the **cheapest edge that connects a new vertex** to the growing tree. Good for **dense** graphs. (Vertex-based.)
- **Kruskal's algorithm:** sort **all edges** by weight; add the smallest edge that **does not form a cycle** (uses **Union-Find / Disjoint Set**). Good for **sparse** graphs. (Edge-based.)

Memory hook: *Prim grows one tree from a vertex; Kruskal picks the cheapest edges globally.*

6.9 Dijkstra's Shortest Path

Dijkstra's algorithm finds the **shortest path from one source vertex to all others** in a **weighted graph with non-negative edges**. It repeatedly picks the **nearest unvisited** vertex and **relaxes** its neighbours (updates a neighbour's distance if going through the current vertex is shorter). Uses a **min-priority queue**; time $O((V+E) \log V)$.

Worked idea (source A, graph above): start $\text{dist}(A)=0$, others $=\infty$. Pick A $\rightarrow$ update B=4, C=2. Pick C (nearest) $\rightarrow$ update B to 3 ($2+1$). Pick B(3) $\rightarrow$ update D=8... continue until all are finalised. Final distances give the shortest path to every node.

Note: Dijkstra fails with **negative** edge weights — use **Bellman-Ford** for those.

6.10 Huffman Coding

Huffman coding is a **lossless compression** method that gives **shorter binary codes to frequent characters** and longer codes to rare ones (variable-length prefix codes), reducing total file size.

Steps:

1. Count the frequency of each character.
2. Put all characters in a **min-heap** (by frequency).
3. Repeatedly **remove the two lowest-frequency nodes** and merge them into a new node whose frequency is their sum; put it back. Repeat until one tree remains.
4. Label left edges **0** and right edges **1**; each character's code is the path from the root to its leaf.

The **prefix property** (no code is a prefix of another) guarantees unambiguous decoding. Used in ZIP, JPEG, MP3.

7. Software Engineering

7.1 SDLC (Software Development Life Cycle)

The idea. The **SDLC** is the organised process a team follows to build software, so the product is reliable and on time.

Phases: Requirement gathering – Analysis – Design – Implementation (coding) – Testing – Deployment – Maintenance.

Models (know the difference and when to use each):

Model	How it works	Best when
Waterfall	Strict linear sequence of phases	Requirements are fixed and clear
V-Model	Waterfall with a matching test stage for each phase	Safety-critical systems
Iterative / Incremental	Build in repeated small pieces	Requirements evolve over time
Spiral	Iterative loops, each with risk analysis	Large, high-risk projects
Agile (Scrum)	Short sprints with continuous feedback	Fast delivery, changing requirements

Waterfall vs Agile (common question). *Waterfall* is rigid and sequential, tests only at the end, and is hard to change once started. *Agile* is flexible and iterative, tests continuously, and welcomes change — it uses **sprints**, **daily stand-ups**, and a **product backlog**. Banks increasingly use Agile for digital-banking apps while keeping strict controls.

7.2 Software Testing

Testing checks that software works correctly and meets requirements.

- **Functional testing** — checks **what** the system does, against the requirements (black-box). Levels, bottom-up: **Unit – Integration – System – Acceptance (UAT)**.
- **Non-functional testing** — checks performance, load, security, usability.
- **Black-box vs White-box:** black-box tests inputs/outputs without seeing the code; white-box tests the internal logic and code paths.
- **Regression testing** — re-run existing tests after a change to make sure nothing that worked before is now broken.
- **Smoke / Sanity testing** — a quick check that the core features work before deeper testing.

Test levels explained: Unit (one function/module), **Integration** (modules working together), **System** (the whole application), **Acceptance** (the customer/user confirms it meets their needs).

7.3 API (Application Programming Interface)

The idea. An **API** is a **contract** that lets two software systems talk to each other. It defines what requests you can make and what responses you get, **without exposing internal code**.

Analogy: a waiter — you order from the menu (request), the kitchen (server) prepares it, the waiter brings your food (response); you never enter the kitchen.

- **REST API** — uses **HTTP methods**: **GET** (read), **POST** (create), **PUT** (update/replace), **PATCH** (partial update), **DELETE** (remove). Data is usually **JSON**; REST is **stateless** (each request is independent).
- **SOAP** — older, XML-based, stricter, with built-in security — still used in some banking systems.
- **HTTP status codes**: **200** OK, **201** Created, **400** Bad Request, **401** Unauthorized, **403** Forbidden, **404** Not Found, **500** Internal Server Error.

Banking angle: **Open Banking / payment gateways** expose secure REST APIs (protected with **OAuth 2.0** tokens) so approved apps can check balances or start transfers safely.

7.4 Version Control (Git)

The idea. **Version control** records every change to your code so a team can work together, review history, and roll back mistakes. **Git** is a **distributed** version-control system — every developer has the full history on their own machine.

Essential commands:

<code>git init</code>	<code># create a new repository</code>
<code>git clone <url></code>	<code># copy a remote repository</code>
<code>git add <file></code>	<code># stage changes</code>
<code>git commit -m "message"</code>	<code># save a snapshot with a message</code>
<code>git push</code>	<code># upload commits to the remote (e.g. GitHub)</code>
<code>git pull</code>	<code># download and merge remote changes</code>
<code>git branch feature</code>	<code># create a branch</code>
<code>git checkout -b feature</code>	<code># create and switch to a branch</code>
<code>git merge feature</code>	<code># combine a branch into the current one</code>
<code>git status / git log</code>	<code># see state / history</code>

Key terms: **repository** (project + history), **commit** (a saved snapshot), **branch** (a parallel line of work), **merge** (combine branches), **pull request** (propose merging your changes), **conflict** (two people changed the same lines). **Git vs GitHub:** Git is the **tool**; GitHub is a **website that hosts** Git repositories.

8. Emerging Technologies & Infrastructure

8.1 Cloud Computing — Service Models

The idea. Cloud computing delivers computing resources — servers, storage, databases, software — **over the internet, on demand, pay-as-you-go**, instead of owning physical machines.

Service models (the “pizza-as-a-service” analogy):

Model	You manage	Provider manages	Example
IaaS (Infrastructure)	OS, apps, data	Servers, storage, network	AWS EC2, Azure VM
PaaS (Platform)	Your app + data	OS, runtime, servers	Google App Engine, Heroku
SaaS (Software)	Nothing — just use it	Everything	Gmail, Office 365, Salesforce

Memory hook: IaaS = rent the kitchen; PaaS = rent kitchen + tools; SaaS = order the finished pizza.

8.2 Cloud — Deployment Models

- **Public cloud:** shared infrastructure owned by a provider (AWS, Azure) — cheap and highly scalable.
- **Private cloud:** dedicated to one organisation — more control and security (banks favour this for sensitive core systems).
- **Hybrid cloud:** a mix — keep sensitive data private, use public cloud for extra capacity (“cloud bursting”).
- **Community cloud:** shared by several organisations with common requirements.

Benefits: scalability, elasticity, cost efficiency, high availability. **Bank concerns:** data sovereignty, regulatory compliance, and security.

8.3 Virtualization, VM & Bare-Metal

Virtualization lets **one physical machine run several isolated virtual machines**, using a **hypervisor** to share the hardware.

- **VM (Virtual Machine):** a software computer with its **own full OS**, running on shared hardware.
- **Hypervisor: Type 1** (bare-metal — runs directly on hardware: VMware ESXi, Hyper-V; fast, for data centres) vs **Type 2** (hosted — runs on top of a normal OS: VirtualBox; for desktops).
- **Bare-metal server:** a **single-tenant physical server** with **no hypervisor** — full hardware performance, dedicated to one customer (used for high-performance databases).

- **Containers (Docker):** lighter than VMs — they **share the host OS kernel** and package only the app + its dependencies; started in seconds and orchestrated by **Kubernetes**.

VM vs Container: a VM virtualises the **hardware** (heavy, has its own OS); a container virtualises the **OS** (lightweight, shares the kernel, starts fast).

8.4 Data Centre Infrastructure

A **data centre** houses an organisation's IT equipment. Core components: **servers, storage (SAN / NAS), networking gear, power** (UPS + backup generators), **cooling**, and **physical security**.

- **Cooling systems** keep hardware within safe temperatures: **CRAC/CRAH units, hot-aisle / cold-aisle** containment, chilled-water and precision air-conditioning. Overheating causes failures, so cooling is mission-critical.
- **Redundancy levels: N+1** (one spare unit beyond the requirement) and **2N** (a full duplicate) for power and cooling. **Tier I–IV** (Uptime Institute) rate reliability; **Tier IV** is fully fault-tolerant at about **99.995%** uptime.

8.5 Backup, Disaster Recovery & SLA (bank-critical)

- **Backup:** copies of data kept so you can restore after loss. **Full** (everything), **Incremental** (only changes since the last backup — small and fast), **Differential** (changes since the last full backup). **3-2-1 rule:** keep **3** copies, on **2** different media, with **1** off-site.
- **Disaster Recovery (DR):** the plan + infrastructure to restore operations after a disaster (fire, flood, cyber-attack). Key metrics:
 - **RTO (Recovery Time Objective):** how **quickly** you must be back up (maximum acceptable downtime).
 - **RPO (Recovery Point Objective):** how much **data loss** is acceptable (how far back the last usable backup is).
 - A **DR site** can be **hot** (live, instant switchover), **warm** (partly ready), or **cold** (bare, slow to bring up).
- **SLA (Service Level Agreement):** a contract that promises a **service level** — **uptime %** (e.g. 99.9% ≈ 8.7 hours downtime/year), response times, and penalties if missed. Banks demand strict high-availability SLAs from their IT and cloud vendors.

Memory hook: **RTO = Time to recover; RPO = the Point (amount of data) you can afford to lose.**

8.6 Other Buzzwords (each worth a line)

- **Blockchain:** a distributed, tamper-evident ledger; each block links to the previous one via a hash (crypto-currencies, secure records).
- **AI / Machine Learning:** systems that learn patterns from data — used by banks for **fraud detection** and **credit scoring**.
- **IoT (Internet of Things):** everyday physical devices connected to the internet (sensors, smart ATMs).

- **Big Data — the 5 Vs:** Volume, Velocity, Variety, Veracity, Value.
- **Edge computing:** process data near where it is generated to cut latency.

9. Last-48-Hours Rapid Revision Sheet

Number systems — binary place values 128-64-32-16-8-4-2-1 · group 3 bits = octal, 4 bits = hex · 2's complement = **flip + add 1** · 8-bit range -128...127.

Logic — NAND & NOR are **universal** · XOR = "inputs differ" · De Morgan $(AB)' = A' + B'$, $(A+B)' = A' B'$ · MUX: n select lines → 2^n inputs · Half adder: Sum = $A \oplus B$, Carry = $A \cdot B$ · Full adder = 2 half adders + OR.

OS — Process = own memory; Thread = shared memory · **Coffman conditions**: Mutual exclusion, Hold & wait, No preemption, Circular wait · TAT = CT + AT; WT = TAT - BT · SJF gives minimum average waiting time · Banker's = find a safe sequence · Paging = fixed pages (internal frag); Segmentation = variable segments (external frag) · avg rotational latency = half a rotation.

DBMS — 1NF atomic, 2NF no partial dependency, 3NF no transitive dependency · **ACID** = Atomicity, Consistency, Isolation, Durability · Index = faster reads, slower writes (B-tree) · Trigger fires automatically on an event · **RAID** 0 stripe, 1 mirror, 5 parity, 10 mirror+stripe.

Networking — OSI = All People Seem To Need Data Processing · TCP reliable / UDP fast · Ports: DNS 53, HTTP 80, HTTPS 443, SMTP 25, POP3 110, IMAP 143 · Switch = L2 (MAC), Router = L3 (IP) · Subnet: hosts = $2^h - 2$, subnets = 2^{borrowed} , block = $256 - \text{mask}$ · Private ranges 10/8, 172.16/12, 192.168/16.

Security — CIA = Confidentiality, Integrity, Availability · Hash = one-way; Encryption = two-way · Symmetric (AES, one key) vs Asymmetric (RSA, key pair) · **IDS detects, IPS prevents** · VPN = encrypted tunnel · Steganography = hide data inside files.

Programming — break = exit loop, continue = skip iteration · &x = address, *p = value at address · recursion needs a base case · overloading = compile-time (same class); overriding = run-time (child class) · OOP pillars = Encapsulation, Inheritance, Polymorphism, Abstraction.

DSA — Stack LIFO, Queue FIFO · BFS = Queue, DFS = Stack · Inorder of a BST = sorted order · Merge sort always $O(n \log n)$; Quick sort worst $O(n^2)$ · Prim (vertex) & Kruskal (edge) build an MST · Dijkstra = shortest path with non-negative weights · Huffman = frequent characters get shorter codes.

Software Engineering — SDLC = Requirement → Design → Code → Test → Deploy → Maintain · Waterfall rigid, Agile flexible (sprints) · REST verbs GET/POST/PUT/DELETE · Git = distributed VCS (add → commit → push).

Cloud / Infra — IaaS / PaaS / SaaS (rent kitchen / +tools / finished pizza) · Public / Private / Hybrid · VM = own OS (heavy), Container = shares kernel (light) · Backup 3-2-1 · **RTO = recovery time, RPO = data-loss point** · SLA = uptime promise.

Exam-day strategy

1. **Attempt every question** — a 10-mark question with a partial answer still scores; never leave a blank.

2. **Structure each answer:** definition → simple explanation → **diagram/example** → banking relevance. Use short headings/bullets so the examiner finds your points fast.
3. **For math** (scheduling, subnetting, number systems, disk): show **every step** and **box the final answer** — method carries marks.
4. **Draw the diagram** for OSI, adders, process states, ER, BST, TCP handshake — quick, guaranteed marks.
5. **Time management:** do the questions you know cold **first**, then the math, then theory. Keep the last few minutes to review.

All the best for 25 July — go in calm and confident. You have prepared well.

10. Predicted Question Bank (50 questions with answers)

These are the questions most likely to appear in the 15 × 10-mark subject section, based on the verified pattern of past Bank IT written papers. Practise writing each as a full structured answer.

A. Computer Architecture & Digital Electronics

1. **Convert $(156)_{10}$ to binary, octal and hexadecimal.** → $10011100_2, 234_8, 9C_{16}$.
2. **Represent -45 in 8-bit 2's complement and compute $30 - 45$.** → $-45 = 11010011$; result = $11110001 = -15$.
3. **What is 2's complement and why do computers use it?** → Flip bits + add 1; it unifies add/subtract in one circuit and has a single zero.
4. **Design a full adder: truth table, Sum & Carry expressions, circuit.** → Sum = $A \oplus B \oplus C_{in}$, Cout = $AB + C_{in}(A \oplus B)$; two half adders + OR gate.
5. **What are universal gates? Build a NOT and an AND using only NAND.** → NAND & NOR; NOT = NAND(A,A); AND = NAND then NOT.
6. **Simplify $F(A,B,C) = \Sigma(1,3,5,7)$ using a K-map.** → $F = C$ (all minterms have $C = 1$).
7. **What is a multiplexer? How many select lines does a 16-to-1 MUX need?** → Data selector; 4 select lines ($2^4 = 16$).
8. **State De Morgan's theorems with proof by truth table.** → $(AB)' = A' + B'$ and $(A+B)' = A'B'$.
9. **Explain the memory hierarchy from registers to disk.** → Registers → Cache → RAM → SSD/HDD; speed & cost/bit fall, capacity rises.
10. **Differentiate RISC vs CISC and Von Neumann vs Harvard.** → RISC simple/fixed vs CISC complex; Von Neumann shared memory vs Harvard separate.

B. Operating Systems

11. **Difference between a process and a thread (with example).** → Process = own memory; thread = shared memory within a process; e.g. browser vs its tabs.
12. **Given 4 processes, draw Gantt charts for FCFS and SJF and find average WT & TAT.** → Tabulate CT, $TAT = CT - AT$, $WT = TAT - BT$; SJF gives lower average WT.
13. **What is a deadlock? State the four Coffman conditions.** → Mutual exclusion, Hold & wait, No preemption, Circular wait.
14. **Explain Banker's Algorithm with a worked safe-sequence example.** → Need = Max - Alloc; pick a process with $Need \leq Available$, run, release; e.g. $\langle P_1, P_3, P_4, P_0, P_2 \rangle$.
15. **Differentiate deadlock, livelock and starvation.** → Deadlock = stuck circular wait; livelock = active but no progress; starvation = indefinite waiting, fixed by aging.
16. **Explain paging vs segmentation with a diagram.** → Fixed pages (internal frag) vs variable segments (external frag); show address translation.
17. **What is virtual memory? Explain page faults and thrashing.** → Illusion of large memory via disk swap; fault = page not in RAM; thrashing = excessive swapping.
18. **Round Robin scheduling with quantum = 4 for given bursts — find WT & TAT.** → Slice each process by the quantum, cycle through, then compute averages.

19. **Disk scheduling: compute total head movement for FCFS and SSTF.** → Sum absolute head moves; SSTF services the nearest request each time.
20. **Explain context switching and the role of the PCB.** → Save one process's PCB, load another's; PCB stores PID, state, PC, registers.

C. Database Management

21. **Explain the ACID properties with a banking example.** → Atomicity, Consistency, Isolation, Durability; illustrate with a fund transfer.
22. **Normalize a given un-normalized table up to 3NF.** → 1NF atomic; 2NF remove partial deps; 3NF remove transitive deps; list final tables.
23. **Draw an ER diagram for a bank (Customer, Account, Loan) and mark keys & cardinality.** → Entities, attributes (underline PK), relationships, 1:N / M:N.
24. **What is indexing? Clustered vs non-clustered, and why banks use it.** → B-tree lookups; clustered sorts rows (one per table), non-clustered many; speeds queries on acc_no.
25. **Write SQL: second-highest balance; total balance per branch; a join.** → `SELECT MAX(balance) ... < (SELECT MAX(balance)...); GROUP BY branch; INNER JOIN on acc_no.`
26. **Difference between a stored procedure and a trigger.** → Procedure is called explicitly; trigger fires automatically on INSERT/UPDATE/DELETE.
27. **Explain the RAID levels 0, 1, 5, 10 and which a bank should use.** → Stripe/mirror/parity/mirror+stripe; banks use 5, 6 or 10.
28. **What are primary, candidate and foreign keys?** → Candidate = minimal unique; primary = chosen candidate; foreign = reference to another PK.
29. **DDL vs DML vs DCL vs TCL with examples.** → CREATE/ALTER; SELECT/INSERT; GRANT/REVOKE; COMMIT/ROLLBACK.
30. **What is denormalization and when is it useful?** → Add redundancy to speed reads; used in reporting/data-warehouse.

D. Networking & Security

31. **Explain the OSI 7-layer model with the function of each layer.** → Application... Physical; give one protocol/device per layer; mnemonic APSTNDP.
32. **Subnet 192.168.10.0/24 into 4 subnets — mask, hosts, network & broadcast per subnet.** → /26, mask 255.255.255.192, 62 hosts, block 64; list all four ranges.
33. **Difference between TCP and UDP; explain the 3-way handshake.** → Reliable/ordered vs fast/connectionless; SYN → SYN-ACK → ACK.
34. **CRC: data 1101011011, generator 10011 — find the CRC and transmitted frame.** → Append 4 zeros, XOR-divide, remainder = 1110; frame = 1101011011 1110.
35. **Symmetric vs asymmetric cryptography; how does HTTPS use both?** → One key vs key pair; asymmetric exchanges a symmetric session key, then symmetric encrypts data.
36. **Explain the CIA triad with an example of each.** → Confidentiality (encryption), Integrity (hashing), Availability (backups/DDoS protection).

37. **Difference between IDS and IPS; what is a firewall?** → IDS detects & alerts (passive); IPS blocks (inline); firewall filters by rules.
38. **Shannon's theorem: 3000 Hz bandwidth, 30 dB SNR — maximum data rate.** → $S/N = 1000$; $C = 3000 \log_2(1001) \approx 30$ kbps.
39. **Explain hashing vs encryption; why store password hashes with a salt?** → One-way vs two-way; salt defeats rainbow-table/precomputed attacks.
40. **Define phishing, ransomware, MITM, SQL injection and one mitigation each.** → Awareness/filters; backups; TLS; parameterised queries.

E. Programming & DSA

41. **Write a complete C program to check whether a number is prime.** → Loop i from 2 to $\sqrt{n}$; if any divides n , not prime (see Program 1).
 42. **Write a complete C program for Fibonacci (iterative and recursive).** → Iterative with two variables; recursive $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$ (Programs 3 & 17).
 43. **Write a complete C program to sort an array using bubble sort.** → Nested loops swapping adjacent elements (Program 12).
 44. **Explain the four pillars of OOP with a banking example.** → Encapsulation (private balance), Inheritance (SavingsAccount), Polymorphism, Abstraction (ATM).
 45. **Overloading vs overriding with code examples.** → Same name different params (compile-time) vs child redefines parent method (run-time).
 46. **Convert the infix expression $(A + B) * (C - D)$ to postfix.** → $A B + C D - *$.
 47. **Explain BFS and DFS with the data structure each uses and one use-case.** → BFS = Queue (shortest path); DFS = Stack (cycle detection).
 48. **Compare Prim's and Kruskal's algorithms for MST.** → Vertex-based greedy vs edge-based with Union-Find; dense vs sparse graphs.
 49. **Explain Dijkstra's algorithm and its limitation.** → Greedy shortest path from a source; fails with negative weights (use Bellman-Ford).
 50. **Compare the time complexity of bubble, merge and quick sort.** → $O(n^2)$ vs $O(n \log n)$ vs $O(n \log n)$ avg / $O(n^2)$ worst; note stability and space.
-

End of guide. Revise the rapid sheet (Section 9) the night before, and rewrite 4–5 of the complete programs from memory so your hand remembers them. Good luck.